Projeto Synthetic Ledger Dash UI

Relatório Técnico de Benchmark de Backends para Dashboards Gerenciais em Tempo Real

Avaliação arquitetural, operacional e experimental de ClickHouse, Druid, Pinot e Materialize no repositório Synthetic Ledger Dash UI

Ambiente local instrumentado

2026-03-09

Projeto Synthetic Ledger Dash UI

Relatório Técnico de Benchmark de Backends para Dashboards Gerenciais em Tempo Real

Avaliação arquitetural, operacional e experimental de ClickHouse, Druid, Pinot e Materialize no repositório Synthetic Ledger Dash UI

Análise do benchmark de convergência visual, corretude contábil e latência percebida em painéis orientados por eventos

Repositório Synthetic Ledger Dash UI

Ambiente local instrumentado
2026-03-09

RESUMO

Este relatório analisa como diferentes famílias de backends analíticos e incrementais se comportam quando usadas como camada de serving para painéis gerenciais near-real-time orientados por eventos. O problema avaliado não se reduz à latência SQL isolada; ele envolve a distância entre o evento canônico de negócio e a disponibilidade de um estado derivado semanticamente e contabilmente confiável no frontend. Para enfrentar esse problema, o documento consolida um benchmark reprodutível sobre um repositório monolítico que integra producer, OpenTelemetry Collector, Kafka, storage writer, APIs especializadas, gateway realtime e frontends por backend. O corpus tecnológico comparado abrange ClickHouse, Apache Druid, Apache Pinot e Materialize, cobrindo tanto motores hot-analytic quanto sistemas incremental-streaming. A metodologia experimental opera por rodadas individualizadas, com coleta de métricas de bootstrap, latência de API, latência SQL, convergência websocket, corretude contábil e consumo de recursos. As tabelas do capítulo de resultados são geradas automaticamente a partir dos artefatos persistidos em `artifacts/benchmark`, preservando rastreabilidade entre coleta, síntese tabular e interpretação analítica.

Palavras-chave: benchmark de backends; dashboards em tempo real; event sourcing; OLAP; incremental view maintenance; Materialize; ClickHouse; Druid; Pinot.

ABSTRACT

This report analyzes how different analytical and incremental backends behave when used as the serving layer for event-driven near-real-time management dashboards. The evaluated problem is not limited to isolated SQL latency; it concerns the distance between the canonical business event and the availability of a semantically and financially reliable derived state in the frontend. To address this problem, the document consolidates a reproducible benchmark on top of a monolithic repository integrating producer, OpenTelemetry Collector, Kafka, storage writer, specialized APIs, a realtime gateway and backend-specific frontends. The compared technological corpus includes ClickHouse, Apache Druid, Apache Pinot and Materialize, therefore covering both hot-analytic engines and incremental-streaming systems. The experimental methodology operates through isolated rounds with metrics for bootstrap, API latency, SQL latency, websocket convergence, accounting correctness and resource consumption. The results chapter tables are generated automatically from the persisted artifacts in `artifacts/benchmark`, preserving traceability between collection, tabular synthesis and analytical interpretation.

Keywords: backend benchmarking; real-time dashboards; event sourcing; OLAP; incremental view maintenance; Materialize; ClickHouse; Druid; Pinot.

NOTA DE ESCOPO

Este pacote organiza em uma única pasta os arquivos-fonte do relatório, o template ABNT de compilação, o capítulo de resultados gerado a partir dos artefatos e o PDF resultante, sempre com base no estado observável do benchmark implementado no repositório.

CRITÉRIOS EDITORIAIS

O texto adota formato impessoal, ancora afirmações factuais em artefatos verificáveis do repositório e evita substituir dados coletados por valores exemplificativos.

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Contextualização do problema . . . 13
1.2 Problema analítico . . . 13
1.3 Premissas de avaliação . . . 13
1.4 Objetivos . . . 14
1.5 Justificativa . . . 14
1.6 Estrutura do relatório . . . 15

2 FUNDAMENTAÇÃO TEÓRICA . . . 16
2.1 Delimitação do problema científico . . . 16
2.2 Event sourcing, ledger canônico e corretude contábil . . . 16
2.3 Sistemas push, pull e arquiteturas híbridas de convergência visual 17
2.4 OLAP quente, HTAP e sistemas relacionais orientados a stream 18
2.5 Incremental view maintenance, retrações e joins incrementais . 19
2.6 Latência percebida, continuidade visual e utilidade gerencial . . 19
2.7 Validade experimental em sistemas distribuídos locais . . . 20
2.8 Síntese teórica do capítulo . . . 21
2.9 Bibliografia-base do Capítulo 2 . . . 21

3 ESTADO DA ARTE TECNOLÓGICO . . . 22
3.1 Introdução . . . 22
3.2 ClickHouse . . . 22

3.2.1 Classe arquitetural . . . 22
3.2.2 Internals relevantes . . . 22
3.2.3 Ingestão e serving no caso de uso . . . 23
3.2.4 Limites e riscos . . . 23
3.2.5 Bibliografia-base de ClickHouse . . . 24
3.3 Apache Druid . . . 24
3.3.1 Classe arquitetural . . . 24
3.3.2 Internals relevantes . . . 24
3.3.3 Druid no contexto de dashboards push-oriented . . . 25
3.3.4 Limites e riscos . . . 25
3.3.5 Bibliografia-base de Druid . . . 25
3.4 Apache Pinot . . . 25
3.4.1 Classe arquitetural . . . 25
3.4.2 Internals relevantes . . . 26
3.4.3 Estado da literatura e fontes técnicas . . . 26
3.4.4 Pinot no contexto do benchmark . . . 26
3.4.5 Limites e riscos . . . 27
3.4.6 Bibliografia-base de Pinot . . . 27
3.5 Materialize . . . 27
3.5.1 Classe arquitetural . . . 27
3.5.2 Fundamentos internos . . . 27
3.5.3 Particularidades semânticas e custos cognitivos . . . 28
3.5.4 Materialize no caso de uso do repositório . . . 28
3.5.5 Limites e riscos . . . 29
3.5.6 Bibliografia-base de Materialize . . . 29
3.6 Síntese comparativa entre categorias . . . 29
3.7 Conclusão do capítulo . . . 29
3.8 Bibliografia-base do Capítulo 3 . . . 30

4 MODELO DO PROBLEMA E ARQUITETURA EXPERIMENTAL . . . 31
4.1 Domínio funcional . . . 31
4.2 Contrato canônico dos dados . . . 31
4.3 Arquitetura do repositório . . . 32
4.4 Estratégias de convergência visual por backend . . . 32
4.5 Observabilidade e pontos de controle . . . 32
4.6 Consequências metodológicas . . . 33

5 METODOLOGIA DE BENCHMARK . . . 34
5.1 Tipo de pesquisa . . . 34
5.2 Unidade experimental . . . 34

5.3 Janela temporal da rodada **34**
5.4 Fluxo operacional da coleta **35**
5.4.1 Preparação 35
5.4.2 Bootstrap 35
5.4.3 Warm-up 35
5.4.4 Coleta principal 36
5.4.5 Encerramento 36
5.5 Métricas nucleares **36**
5.5.1 Bootstrap e disponibilidade 36
5.5.2 Latência de API 36
5.5.3 Latência SQL nativa 36
5.5.4 Convergência visual no frontend 36
5.5.5 Corretude contábil 37
5.5.6 Recursos computacionais 37
5.6 Artefatos canônicos da rodada **37**
5.7 Implementação concreta no repositório **38**
5.8 Ameaças à validade **38**
5.9 Fecho metodológico **38**

6 RESULTADOS COMPARATIVOS DA BATERIA CONCLUSIVA . . . **39**
6.1 Objetivo desta seção **39**
6.2 Corpus empírico utilizado **39**
6.3 Métricas consolidadas comparativas **39**
6.4 Análise por dimensão observada **40**
6.4.1 Latência de API 40
6.4.2 Latência SQL nativa 40
6.4.3 Convergência visual e prontidão percebida 40
6.4.4 Corretude contábil e estabilidade operacional 41
6.5 Síntese interpretativa **41**
6.6 Limites desta evidência **42**

7 RESULTADOS COMPARATIVOS DERIVADOS DOS ARTEFATOS . **43**
7.1 Corpus, seleção canônica e rastreabilidade **43**
7.2 Latência HTTP e SQL **46**
7.3 Convergência visual e experiência percebida **47**
7.4 Estabilidade operacional e cobertura diagnóstica **47**
7.5 Recursos computacionais **50**
7.5.1 Backend Clickhouse 50
7.5.2 Backend Druid 52
7.5.3 Backend Materialize 54

7.5.4 Backend Pinot 56
7.6 Síntese orientada pelos dados 58

8 ANÁLISE TÉCNICA POR BACKEND 59
8.1 Propósito deste capítulo 59
8.2 ClickHouse sob o protocolo do experimento 59
8.2.1 Estrutura funcional da trilha ClickHouse 59
8.2.2 Superfície HTTP observada 60
8.2.3 SQL nativo e leitura quente 60
8.2.4 Convergência visual e relação com o gateway 61
8.2.5 Diagnóstico da tentativa substituída 61
8.3 Druid sob o protocolo do experimento 61
8.3.1 A complexidade da topologia Druid no corpus 61
8.3.2 Warm-up e transição de readiness 62
8.3.3 A anomalia de `filtered_entries` 62
8.3.4 Economia de experiência percebida 63
8.3.5 Leitura crítica da trilha Druid 63
8.4 Pinot sob o protocolo do experimento 63
8.4.1 Superioridade consultiva e seus limites 63
8.4.2 O papel do frontend especializado 64
8.4.3 Ausência de cobertura diagnóstica no fim da rodada 64
8.4.4 Leitura arquitetural final de Pinot 64
8.5 Materialize sob o protocolo do experimento 65
8.5.1 O backend mais teoricamente promissor do corpus 65
8.5.2 Dissociação entre utilidade inicial e custo consultivo 65
8.5.3 Volatilidade e cobertura de debug 65
8.5.4 O valor científico da trilha Materialize 66
8.6 Síntese técnica do capítulo 66

9 DISCUSSÃO COMPARATIVA AMPLIADA 67
9.1 Por que a tabela-resumo não basta 67
9.2 Dimensão 1: latência de consulta não é latência de sistema . . . 67
9.3 Dimensão 2: estado quente versus estado incremental 68
9.4 Dimensão 3: prontidão percebida, autoridade e confiança visual 68
9.5 Dimensão 4: custo operacional e custo cognitivo são métricas reais 69
9.6 Dimensão 5: corretude contábil como fronteira mínima, não como diferencial suficiente 69
9.7 Dimensão 6: o peso do corpus auxiliar e das tentativas substituídas 70

9.8 Dimensão 7: relação entre recursos e desempenho 70
9.9 Uma matriz interpretativa para decisão arquitetural 71
9.10 Encerramento da discussão comparativa 71

10 IMPLICAÇÕES DE ENGENHARIA E PROGRAMA EXPERIMENTAL 72
10.1 O benchmark como ferramenta de arquitetura, não apenas de medição 72
10.2 Contratos canônicos como patrimônio do experimento 72
10.3 Observabilidade como requisito de comparabilidade 73
10.4 Replays, correções tardias e semântica temporal 73
10.5 Engenharia de frontend e o erro de subestimar a borda 74
10.6 Uma agenda objetiva para reruns de dupla conformação 74
10.7 Expansão do protocolo: além de uma única rodada canônica . . 75
10.8 O papel do relatório ampliado dentro do repositório 75
10.9 Fecho do programa experimental 75

11 MATRIZ DE DECISÃO ARQUITETURAL 77
11.1 Por que este capítulo existe 77
11.2 Eixo decisório 1: quando a prioridade dominante é consulta rápida de painel 77
11.3 Eixo decisório 2: quando a prioridade dominante é prontidão percebida do frontend 78
11.4 Eixo decisório 3: quando a prioridade dominante é governança do estado autoritativo 79
11.5 Eixo decisório 4: quando a prioridade dominante é simplicidade operacional local 79
11.6 Eixo decisório 5: quando a prioridade dominante é aprendizagem estratégica da organização 80
11.7 Cenários de decisão sintetizados 80
11.8 O que a matriz de decisão corrige em relação a benchmarks simplificados 81
11.9 Cenários exemplares de decisão 81
11.10 Trade-offs que não devem ser omitidos no uso do relatório 82
11.11 Fecho da matriz decisória 83

12 CONCLUSÃO 84
12.1 Resposta à pergunta do relatório 84
12.2 Contribuição analítica principal 84
12.3 Implicações arquiteturais 84
12.4 Limites e agenda imediata 85

12.5 Fecho final . . . 85

A APENDICE A. PROTOCOLO DE COLETA . . . 86
A.1 Proposito . . . 86
A.2 Principios . . . 86
A.3 Fluxo padrao . . . 86
A.4 Scripts implementados . . . 86
A.5 Observacao metodologica . . . 87

B APENDICE B. ESPECIFICACAO CANONICA DOS ARTEFATOS . . . 88
B.1 Artefatos obrigatorios . . . 88
B.2 Estrutura logica do consolidado . . . 88
B.3 Colunas transversais do CSV . . . 88
B.4 Regra semantica . . . 89

C APÊNDICE C - DOSSIÊS OPERACIONAIS POR BACKEND . . . 90
C.1 Finalidade dos dossiês . . . 90
C.2 Dossiê ClickHouse . . . 90
C.2.1 Papel da stack dentro do repositório . . . 90
C.2.2 O que a stack parece fazer muito bem . . . 90
C.2.3 O que a stack exige da engenharia . . . 91
C.2.4 Interpretação prática . . . 91
C.3 Dossiê Druid . . . 91
C.3.1 Papel da stack dentro do repositório . . . 91
C.3.2 O que a stack parece fazer muito bem . . . 92
C.3.3 Fragilidades operacionais registradas . . . 92
C.3.4 Interpretação prática . . . 92
C.4 Dossiê Pinot . . . 92
C.4.1 Papel da stack dentro do repositório . . . 92
C.4.2 O que a stack parece fazer muito bem . . . 93
C.4.3 O ponto de tensão da trilha . . . 93
C.4.4 Interpretação prática . . . 93
C.5 Dossiê Materialize . . . 93
C.5.1 Papel da stack dentro do repositório . . . 93
C.5.2 O que a stack parece fazer de forma singular . . . 94
C.5.3 O que limita a maturidade prática da trilha . . . 94
C.5.4 Interpretação prática . . . 94
C.6 Síntese dos dossiês . . . 94

D APÊNDICE D - GLOSSÁRIO ANALÍTICO DE MÉTRICAS, COMPONENTES E PARADIGMAS . . . 95
D.1 Razão deste glossário expandido . . . 95
D.2 Evento canônico . . . 95
D.3 Ledger . . . 95
D.4 Dupla entrada . . . 96
D.5 Read model . . . 96
D.6 Hot-analytic . . . 96
D.7 Incremental-streaming . . . 96
D.8 Incremental view maintenance . . . 97
D.9 Retratação . . . 97
D.10 Snapshot autoritativo . . . 97
D.11 Continuidade visual . . . 97
D.12 `frontend_time_to_first_meaningful_state_ms` . . . 98
D.13 `snapshot_rate_per_second` . . . 98
D.14 `entry_rate_per_second` . . . 98
D.15 Health timeline . . . 98
D.16 Round canônica . . . 98
D.17 Rodada substituída . . . 99
D.18 Rodada de validação . . . 99
D.19 Custo operacional . . . 99
D.20 Custo cognitivo . . . 99
D.21 Sistema completo . . . 100
D.22 Encerramento do glossário . . . 100

E APÊNDICE E - PROGRAMA DE RERUNS, CENÁRIOS E EXTENSÕES DO BENCHMARK . . . 101
E.1 Objetivo deste apêndice . . . 101
E.2 Família de reruns 1: dupla conformação das rodadas canônicas 101
E.3 Família de reruns 2: variação controlada de cardinalidade e payload . . . 102
E.4 Família de reruns 3: replay e reprocessamento . . . 102
E.5 Família de reruns 4: correções tardias e retratações . . . 102
E.6 Família de reruns 5: robustez de observabilidade e completude de debug . . . 103
E.7 Família de cenários 6: degradação parcial e falhas localizadas . 103
E.8 Família de cenários 7: evolução do papel do frontend . . . 104
E.9 Como relatar esses cenários sem perder auditabilidade . . . 104
E.10 Checklist operacional para reruns canônicos . . . 104

E.11 Estratégia de publicação incremental do corpus futuro 105
E.12 Fecho do apêndice 105

LISTA DE TABELAS

Tabela 4 – Inventário completo das rodadas disponíveis no corpus de benchmark. 44
Tabela 5 – Orçamento de fases das rodadas canônicas utilizadas na comparação principal. 45
Tabela 6 – Latência HTTP por endpoint nas rodadas canônicas. 46
Tabela 7 – Latência SQL por consulta sintética nas rodadas canônicas. 46
Tabela 8 – Tempos até o primeiro sucesso observável de API e SQL nas rodadas canônicas. 46
Tabela 9 – Métricas de convergência visual e propagação de eventos nas rodadas canônicas. 47
Tabela 10 – Prontidão observada pelos health checks durante as rodadas canônicas. 47
Tabela 11 – Cobertura dos snapshots finais de debug nas rodadas canônicas. . . 48
Tabela 12 – Falhas efetivamente observadas ao longo de todas as rodadas disponíveis. 49
Tabela 13 – Uso consolidado de recursos computacionais por backend canônico. 50
Tabela 14 – Detalhamento de recursos por papel de serviço para Clickhouse. . . 51
Tabela 15 – Detalhamento de recursos por papel de serviço para Druid. 53
Tabela 16 – Detalhamento de recursos por papel de serviço para Materialize. . . 55
Tabela 17 – Detalhamento de recursos por papel de serviço para Pinot. 57

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO DO PROBLEMA

A digitalização da operação empresarial elevou o papel de arquiteturas orientadas a eventos como espinha dorsal de sistemas transacionais, analíticos e observacionais. Em paralelo, a demanda por painéis gerenciais near-real-time tornou-se estrutural: gestores financeiros, comerciais e operacionais já não aceitam visões consolidadas apenas no fechamento de lote ou em janelas longas de polling. O problema, contudo, não se resolve pela simples exposição de eventos ao frontend. A decisão gerencial depende de um estado derivado consultável, semanticamente coerente e visualmente estável.

Essa exigência torna-se ainda mais severa em domínios contábil-financeiros. Um painel pode parecer rápido e, ainda assim, ser tecnicamente inadequado se violar invariantes de dupla entrada, oscilar entre snapshots inconsistentes ou degradar sob replay e reprocessamento. Logo, o desafio não é apenas acelerar consultas. O desafio é reduzir a distância entre o evento canônico de negócio e a disponibilidade de uma representação útil, autoritativa e contabilmente confiável no frontend.

1.2 PROBLEMA ANALÍTICO

O problema científico desta pesquisa consiste em comparar, de forma metodologicamente justa, backends com paradigmas internos distintos para alimentar painéis gerenciais em tempo real por push. Essa comparação precisa reconhecer tensões simultâneas entre throughput, latência percebida, custo operacional, corretude contábil, capacidade de replay, estabilidade sob filtros e proximidade arquitetural entre stream e estado derivado.

Em termos mais precisos, pergunta-se: como avaliar sistemas hot-analytic e incremental-streaming sob um mesmo contrato funcional, sem reduzir a comparação a um teste superficial de latência HTTP ou SQL? A pergunta é relevante porque diferentes tecnologias oferecem respostas arquiteturais distintas ao mesmo problema de serving, e tais respostas afetam desde a topologia da stack até a experiência visual final.

1.3 PREMISSAS DE AVALIAÇÃO

A premissa central de avaliação é que sistemas de manutenção incremental de views tendem a reduzir a distância entre evento canônico e snapshot derivado consultável, favorecendo modos autoritativos de convergência visual. Entretanto, essa vantagem

potencial vem acompanhada de custos cognitivos, restrições semânticas e desafios operacionais que não aparecem com a mesma intensidade em motores hot-analytic. Em contrapartida, motores analíticos quentes podem preservar excelente desempenho de serving quando o read model já está estabilizado, mas frequentemente exigem coordenação adicional entre API, gateway e frontend para manter coerência perceptual.

1.4 OBJETIVOS

O objetivo geral é estabelecer um benchmark state-of-the-art para avaliar backends capazes de servir painéis contábeis e comerciais near-real-time por push, preservando corretude financeira e convergência visual.

Como objetivos específicos, o trabalho busca:

1. Definir o contrato funcional do caso de uso com foco em ledger canônico, APIs e interface visual.
2. Descrever, implementar e endurecer as trilhas arquiteturais de ClickHouse, Druid, Pinot e Materialize no mesmo repositório experimental.
3. Propor uma taxonomia de métricas que combine latência, corretude, operacionalidade e experiência percebida.
4. Executar rodadas de benchmark reprodutíveis e auditáveis.
5. Interpretar os resultados comparando paradigmas de serving, e não apenas produtos individuais.

1.5 JUSTIFICATIVA

Do ponto de vista industrial, a análise trata de um problema recorrente em plataformas de dados operacionais: como expor indicadores executivos e contábeis quase em tempo real sem transformar o frontend em um mosaico de inconsistências. Do ponto de vista conceitual, o documento articula áreas que costumam aparecer separadas: event sourcing, OLAP quente, streaming relacional, incremental view maintenance e latência percebida em interfaces.

Existe, portanto, uma lacuna entre benchmarks convencionais de banco de dados e a realidade de sistemas gerenciais push-oriented. Este trabalho busca preencher essa lacuna por meio de um protocolo experimental que preserve o sistema completo como unidade de análise.

1.6 ESTRUTURA DO RELATÓRIO

O Capítulo 2 apresenta a fundamentação teórica. O Capítulo 3 discute o estado da arte tecnológico. O Capítulo 4 descreve o modelo do problema e a arquitetura experimental do repositório. O Capítulo 5 formaliza a metodologia de benchmark. O Capítulo 6 registra resultados preliminares concretos já obtidos no ambiente experimental. Por fim, os apêndices consolidam o protocolo de coleta e a especificação canônica dos artefatos.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 DELIMITAÇÃO DO PROBLEMA CIENTÍFICO

O problema investigado nesta pesquisa situa-se na interseção entre sistemas de eventos, modelos de leitura near-real-time, arquitetura de dashboards push-oriented e engenharia de backends analíticos ou incrementais para suporte à decisão gerencial. Não se trata apenas de medir qual sistema responde consultas mais rapidamente em condições laboratoriais simplificadas. O que está em jogo é a capacidade de reduzir, de modo arquiteturalmente defensável, a distância temporal, semântica e operacional entre um evento canônico de negócio e sua tradução em estado derivado consultável por interfaces de decisão.

Em um domínio contábil-financeiro, essa questão ganha gravidade adicional. Ao contrário de workloads puramente observacionais, o fluxo de eventos precisa manter coerência entre granularidade transacional, agregados gerenciais e invariantes de dupla entrada. Isso implica que um backend potencialmente rápido, mas incapaz de preservar consistência entre ledger detalhado, balanço patrimonial e demonstração de resultados sob replay, ajuste ou reprocessamento, não pode ser considerado tecnicamente equivalente a outro que ofereça menor throughput, mas maior robustez semântica. Portanto, a fundamentação teórica desta pesquisa exige uma articulação entre teoria de sistemas orientados a eventos, manutenção incremental de views, arquitetura de bancos analíticos e teoria de desempenho percebido em frontends operacionais.

2.2 EVENT SOURCING, LEDGER CANÔNICO E CORRETUDE CONTÁBIL

Uma primeira base conceitual é o paradigma de event sourcing, no qual os fatos de negócio são registrados como ocorrências imutáveis, ordenadas e auditáveis, e o estado derivado do sistema é reconstruído a partir dessa trilha de eventos. Em sua formulação mais rigorosa, event sourcing não é meramente um padrão de persistência alternativo; ele redefine a relação entre temporalidade, auditabilidade, reprodutibilidade e modelagem do domínio. A vantagem epistêmica do paradigma reside no fato de que o sistema preserva a história do que ocorreu, em vez de apenas armazenar um snapshot tardio do estado vigente.

No caso em estudo, a trilha canônica não se resume ao evento comercial bruto. O repositório organiza a verdade operacional em torno de `ledger-entries-v1`, um stream

canônico de lançamentos contábeis derivados por um `storage_writer` especializado. Essa decisão é teoricamente relevante. Ela introduz uma camada intermediária entre o evento primário e o modelo de leitura, permitindo que o benchmark compare backends sobre uma base financeira normalizada, e não sobre eventos heterogêneos de domínio comercial. Em termos metodológicos, isso reduz o risco de que diferenças de modelagem de ingestão contaminem a comparação entre motores de consulta ou de manutenção incremental.

Em sistemas contábeis, a imutabilidade do ledger é inseparável do princípio de dupla entrada. Cada evento econômico relevante deve ser refletido por pares ou conjuntos coerentes de débito e crédito, de tal modo que a soma agregada das classes patrimoniais e de resultado continue satisfazendo invariantes fundamentais. Essa exigência aproxima o benchmark de uma classe de problemas em que corretude semântica tem precedência sobre latência isolada. Em outras palavras, o sistema não avalia apenas quão rapidamente um painel atualiza, mas quão rapidamente ele atualiza sem degradar a confiabilidade da representação contábil.

Outro conceito central é o de replay. Em arquiteturas baseadas em eventos, replay não é uma exceção operacional, mas uma propriedade estrutural do sistema. Seja por cold start, reprocessamento, mudança de topologia ou recuperação de falha, a capacidade de recomputar modelos de leitura a partir do histórico canônico é uma condição de robustez. Essa propriedade, contudo, afeta de forma desigual backends analíticos e incrementais. Motores de consulta quente tendem a operar melhor quando o read model já foi materializado; motores de manutenção incremental tendem a internalizar parte da lógica de derivação. Assim, a análise teórica do benchmark precisa incluir o replay como variável de primeira classe, e não como mero teste de exceção.

2.3 SISTEMAS PUSH, PULL E ARQUITETURAS HÍBRIDAS DE CONVERGÊNCIA VISUAL

A literatura de sistemas interativos distribuídos distingue, de forma clássica, mecanismos pull, push e modelos híbridos. Em arquiteturas pull, o cliente consulta periodicamente uma fonte autoritativa. O modelo é operacionalmente simples, mas produz um custo recorrente de polling, uma latência perceptual fortemente quantizada pela periodicidade das consultas e uma tensão direta entre responsividade e custo operacional. Em dashboards near-real-time, polling puro tende a impor uma escolha ruim: ou se consulta frequentemente, onerando o sistema, ou se consulta pouco, degradando a atualidade percebida.

Arquiteturas push deslocam parte dessa dinâmica, permitindo que o servidor notifique o cliente quando novos eventos, agregados ou snapshots estiverem dispo-

níveis. No entanto, push puro raramente resolve sozinho o problema de consistência visual. Um stream de eventos unitários pode preservar continuidade perceptual, mas não garante, por si só, que o cliente mantenha uma visão autoritativa de agregados complexos. Isso é particularmente evidente quando o painel depende de joins, filtros compostos, catálogos auxiliares ou agregações contábeis cumulativas. Nesses casos, o push de eventos deve ser articulado com alguma forma de snapshot autoritativo ou de reconciliação incremental.

O repositório estudado concretiza exatamente esse ponto por meio de um `realtime_gateway` que opera como camada de convergência visual. Em algumas stacks, ele preserva um modo `push-first` com projeção local no frontend; em outras, opera com `authoritative snapshot resync`; e, no caso do Materialize, desloca-se para `authoritative incremental snapshot`. Essa taxonomia é teoricamente rica porque mostra que a comparação não é apenas entre bancos, mas entre estratégias de convergência visual acopladas a diferentes modelos de leitura. Em termos de teoria de sistemas interativos, isso significa que a unidade relevante de comparação deixa de ser o banco isolado e passa a ser a combinação banco + API + gateway + frontend.

2.4 OLAP QUENTE, HTAP E SISTEMAS RELACIONAIS ORIENTADOS A STREAM

O problema estudado também se insere na literatura sobre sistemas analíticos. Motores como ClickHouse, Druid e Pinot pertencem, com nuances, a uma família de sistemas desenhados para consultas de baixa latência sobre grandes volumes de dados, frequentemente em colunas, com forte otimização para agregações, filtros e scans seletivos. Ainda que o termo HTAP seja por vezes empregado de forma ampla, convém distinguir sistemas voltados à análise quente de sistemas cuja proposta central é a manutenção incremental do próprio estado derivado.

No contexto deste benchmark, ClickHouse, Druid e Pinot funcionam como motores `hot-analytic`. Eles consomem ou recebem um fluxo canônico, materializam um read model consultável e, a partir daí, servem consultas operacionais com latência baixa. O valor desses sistemas reside em sua eficiência como serving layer analítica. Já o Materialize aproxima-se de outra classe conceitual: sistemas relacionais streaming-first, nos quais a definição de `source`, `view` e `materialized view` participa do próprio mecanismo de convergência incremental entre stream e estado consultável.

Essa distinção é decisiva para a validade do benchmark. Comparar um motor de leitura quente com um sistema de incremental view maintenance como se ambos fossem apenas adaptadores SQL equivalentes produz viés conceitual. O primeiro é otimizado para consultas sobre materialização existente; o segundo tende a reduzir o custo

de recomputação entre evento e view derivada. Portanto, a fundamentação teórica exige reconhecer que a unidade de comparação deve preservar a natureza do paradigma, inclusive no desenho dos cenários, métricas e estratégias de coleta.

2.5 INCREMENTAL VIEW MAINTENANCE, RETRAÇÕES E JOINS INCREMENTAIS

Incremental view maintenance é o campo teórico que investiga como atualizar visões derivadas sem recomputar integralmente o resultado a cada novo fato. Em termos algébricos, o problema consiste em propagar diferenças sobre operadores relacionais e agregações, preservando consistência sem incorrer no custo de recálculo total. Esse tema é central para workloads em que o volume de atualizações é elevado, o custo de recomputação total é proibitivo e a latência de propagação importa diretamente para a utilidade do sistema.

No caso deste trabalho, a manutenção incremental não é uma abstração distante; ela aparece concretamente na necessidade de atualizar `summary`, `workspace`, `sales_workspace`, catálogos e agregados contábeis sob fluxo contínuo de `ledger entries`. Contudo, a teoria adverte que nem toda view é incrementalmente trivial. Agregações não monotônicas, retrações, correções tardias, joins de alta cardinalidade e filtros compostos podem elevar dramaticamente o custo da manutenção incremental. Em sistemas financeiros, isso é particularmente relevante porque devoluções, cancelamentos, ajustes e revisões são semanticamente importantes e frequentemente introduzem retrações no estado derivado.

As retrações merecem atenção especial. Em workloads monotônicos, novas tuplas apenas expandem o resultado. Em workloads com correções, é preciso remover ou invalidar contribuições anteriores. Isso torna a manutenção incremental conceitualmente mais exigente. A literatura sobre Differential Dataflow, por exemplo, mostra como diferenças positivas e negativas podem ser propagadas em dataflows incrementais complexos. Na prática do benchmark, isso significa que métricas de `pending_retractions`, tempo de reconvergência e estabilidade sob replay não são opcionais; elas são a forma empírica de testar se o paradigma incremental sustenta sua promessa sob a complexidade real do domínio.

2.6 LATÊNCIA PERCEBIDA, CONTINUIDADE VISUAL E UTILIDADE GERENCIAL

Uma contribuição importante desta pesquisa é deslocar o foco da latência exclusivamente infraestrutural para a latência percebida no frontend. Em sistemas de dashboard,

a pergunta decisiva não é apenas quando a API ficou apta a responder, mas quando o usuário passou a enxergar um estado útil, coerente e suficientemente atualizado para orientar decisão. Isso aproxima o benchmark da literatura de `time-to-first-meaningful-result`, `responsiveness` e `situational awareness` em sistemas interativos.

Nesse contexto, o conceito de `frontend_time_to_first_meaningful_state_ms` adquire papel teórico. Ele mede o intervalo entre o início da sessão do cliente e o momento em que uma representação minimamente informativa se torna disponível. Esse valor não coincide necessariamente com a primeira resposta HTTP, com o primeiro pacote websocket ou com a primeira linha consultável no banco. Ele representa um limiar semântico: o instante em que o sistema deixa de ser apenas tecnicamente responsivo e passa a ser operacionalmente útil.

Outro conceito associado é o de continuidade perceptual. Um painel pode ser correto do ponto de vista eventual, mas parecer errático, vazio ou incoerente durante a convergência. Portanto, a arquitetura de push deve ser julgada também por sua capacidade de evitar descontinuidades bruscas, listas vazias indevidas, oscilações espúrias de estado e divergência entre visões detalhadas e agregadas. Isso justifica o uso de snapshots autoritativos, projeções locais transitórias ou reconciliações incrementais conforme o backend. Em termos teóricos, a latência percebida é uma propriedade do sistema completo, e não do banco isolado.

2.7 VALIDADE EXPERIMENTAL EM SISTEMAS DISTRIBUÍDOS LOCAIS

Benchmarks de sistemas distribuídos sofrem com problemas clássicos de validade interna e externa. Em ambiente local, jitter de CPU, page cache, efeitos de aquecimento, limites de memória, comportamento do SO hospedeiro e rede Docker podem introduzir variabilidade significativa. Além disso, diferentes backends possuem tempos de bootstrap, formas de aquecimento e estruturas de metadata muito distintas. Comparar resultados sem controlar esses fatores leva a conclusões frágeis.

Por essa razão, o protocolo desta pesquisa adota rodadas individualizadas por backend, com janela máxima de 10 minutos, scripts dedicados, coleta de health timeline, recursos de container, latências de API/SQL e snapshots finais de debug. Esse desenho não elimina todos os vieses, mas melhora substancialmente a comparabilidade. Em particular, a coleta isolada evita que um backend seja penalizado ou favorecido por competir com outros na mesma máquina, o que seria metodologicamente inaceitável quando o objetivo é comparar paradigmas com perfis distintos de consumo.

Também se torna necessário distinguir explicitamente `cold start`, `warm state` e `replay recovery`. Um sistema pode ter excelente latência em estado aquecido e desem-

penho medíocre na primeira leitura útil; outro pode demorar a hidratar, mas oferecer convergência incremental superior após estabilização. Essas diferenças não devem ser apagadas por uma média única. Elas são parte da contribuição analítica do benchmark.

2.8 SÍNTESE TEÓRICA DO CAPÍTULO

Este capítulo estabeleceu que o problema central do benchmark não pode ser reduzido a um duelo simplista de throughput ou tempo de resposta SQL. O caso de uso exige a articulação de cinco eixos teóricos: imutabilidade e replay em sistemas orientados a eventos; corretude contábil como restrição semântica forte; diferenças entre arquiteturas pull, push e híbridas; distinção entre motores analíticos quentes e sistemas de manutenção incremental de views; e latência percebida como fenômeno de sistema completo. A partir dessa base, torna-se possível analisar, no capítulo seguinte, cada tecnologia não como produto isolado, mas como instância de um paradigma com propriedades internas, trade-offs e implicações específicas para dashboards gerenciais near-real-time.

2.9 BIBLIOGRAFIA-BASE DO CAPÍTULO 2

- Abadi, D. J.; Boncz, P. A.; Harizopoulos, S. Column-oriented database systems.
- Abadi, D. J. Query execution in column-oriented database systems.
- Gray, J.; Reuter, A. Transaction Processing: Concepts and Techniques.
- Hellerstein, J. M.; Stonebraker, M.; Hamilton, J. Architecture of a database system.
- Kleppmann, M. Designing Data-Intensive Applications.
- Martin Fowler. Event Sourcing.
- McSherry, F.; Murray, D. G.; Isaacs, R.; Isard, M. Differential Dataflow.
- Murray, D. G.; McSherry, F.; Isaacs, R.; Isard, M.; Barham, P.; Abadi, M. Naiad: A Timely Dataflow System.
- Pat Helland. Immutability Changes Everything.
- Stonebraker, M. et al. C-Store: A Column-oriented DBMS.

3 ESTADO DA ARTE TECNOLÓGICO

3.1 INTRODUÇÃO

O estado da arte relevante para esta pesquisa não pode ser organizado apenas por ordem cronológica de lançamento de produtos ou por comparações superficiais de benchmark industrial. A questão central é identificar como diferentes famílias de sistemas materializam, em sua arquitetura interna, distintas respostas para o mesmo problema: transformar fluxos de eventos em estado derivado consultável com baixa latência, alta confiabilidade e custo operacional aceitável. Nesse sentido, ClickHouse, Druid, Pinot e Materialize não devem ser lidos como variações marginais de uma mesma classe, mas como respostas arquiteturais distintas a tensões entre ingestão, armazenamento, indexação, agregação, invalidação e serving.

Este capítulo apresenta cada backend em profundidade suficiente para sustentar uma comparação científica. O critério de análise não será marketing de produto, mas: arquitetura interna, modelo de armazenamento, caminho de ingestão, estratégia de indexação, semântica de atualização, implicações para dashboards push-oriented, pontos fortes, fragilidades e adequação ao problema do repositório.

3.2 CLICKHOUSE

3.2.1 Classe arquitetural

ClickHouse é um sistema de armazenamento colunar orientado à análise de baixa latência em larga escala. Sua força decorre da combinação entre armazenamento em colunas, compressão agressiva, particionamento por partes, leitura vetorizada e uma família de motores `MergeTree` desenhados para otimizar scans seletivos, filtros e agregações. Em termos de paradigma, ClickHouse opera como um motor `hot-analytic`: ele se torna especialmente eficaz quando o read model relevante já se encontra materializado em tabelas adequadas e quando a consulta pode explorar a localidade estatística e os mecanismos de pruning do armazenamento colunar.

3.2.2 Internals relevantes

No cerne do ClickHouse estão as `parts`, unidades físicas de armazenamento criadas por inserções e posteriormente consolidadas por merges assíncronos. Esse desenho oferece uma relação favorável entre throughput de ingestão e custo de leitura, mas também implica um comportamento temporal específico: o estado consultável pode depender do

ciclo de merges, da materialização de visões auxiliares e da organização da ordenação primária. A indexação é deliberadamente esparsa. Em vez de um B-tree tradicional por linha, ClickHouse aposta em marcas, grânulos e ordenação por chave primária para eliminar grandes faixas de dados com custo baixo.

Essa estrutura é extremamente eficiente para workloads em que filtros e agregações podem ser mapeados sobre colunas bem compressíveis e chaves de ordenação adequadas. No contexto de dashboards gerenciais, isso favorece queries de resumo, slicing por dimensão e leitura de subconjuntos recentes, especialmente quando o fan-out para o ClickHouse já produziu um read model adequado. Em contrapartida, ClickHouse não é, por natureza, um mecanismo de manutenção incremental de views arbitrárias acoplado ao stream em tempo contínuo. Sua força reside mais no serving analítico de um estado quente do que na propagação incremental fina entre evento e view complexa.

3.2.3 Ingestão e serving no caso de uso

No repositório, ClickHouse recebe o fluxo canônico a partir do `storage_writer`, que executa fan-out do ledger contábil para o backend. Isso preserva uma separação clara entre derivação contábil e serving analítico. A vantagem é metodológica e operacional: a lógica financeira central permanece no writer, enquanto ClickHouse se concentra em responder bem sobre uma tabela consultável. Para um benchmark honesto, isso é importante, porque evita atribuir ao banco responsabilidades de semântica de domínio que pertencem ao pipeline canônico.

Em dashboards push-oriented, ClickHouse tende a funcionar melhor com estratégia híbrida: eventos podem sustentar continuidade visual imediata, mas snapshots autoritativos periódicos do gateway oferecem a ancoragem necessária para filtros, agregados e reconciliação. Isso o coloca numa posição forte para consultas de baixa latência, mas menos naturalmente alinhada a snapshots incrementais autoritativos de alta frequência sob filtros compostos altamente dinâmicos.

3.2.4 Limites e riscos

Os limites de ClickHouse, para este caso específico, decorrem menos de desempenho bruto e mais de semântica operacional. O sistema pode responder rapidamente a queries bem ajustadas, mas não internaliza, por si só, uma teoria de retrações e de manutenção incremental de read models comparável à de sistemas como Materialize. Em workloads com alta taxa de correções, replay frequente ou necessidade de snapshots incrementais filtrados muito próximos do stream, a solução tende a exigir mais coordenação na API e no gateway.

3.2.5 Bibliografia-base de ClickHouse

- Stonebraker, M. et al. C-Store: A Column-oriented DBMS.
- Abadi, D. J.; Madden, S.; Ferreira, M. C. Integrating compression and execution in column-oriented database systems.
- ClickHouse, Inc. Documentação técnica oficial do ClickHouse.
- Publicações técnicas e blogs de engenharia do projeto ClickHouse sobre MergeTree, skipping indexes, projections e materialized views.

3.3 APACHE DRUID

3.3.1 Classe arquitetural

Apache Druid é um sistema analítico distribuído fortemente orientado à ingestão de eventos e serving OLAP de baixa latência, especialmente em cenários de séries temporais, exploração interativa e agregações multidimensionais. Diferentemente de um banco monolítico, sua arquitetura explicita uma topologia de serviços com papéis distintos: `Overlord` para coordenação de tarefas de ingestão, `Coordinator` para gestão de segmentos, `Broker` para planejamento e fan-out de consultas, `Router` para front-door HTTP e `Historicals` ou `MiddleManagers` para armazenamento e processamento de dados.

3.3.2 Internals relevantes

Druid organiza dados em segmentos imutáveis, particionados temporalmente e distribuíveis entre nós. A ingestão streaming via Kafka supervisor alimenta tarefas que consomem o stream, constroem segmentos e os publicam para serving. Esse desenho tem vantagens claras: baixa latência para agregações sobre dados temporalmente organizados, boa escalabilidade horizontal e forte adequação a cenários exploratórios com filtros e group-bys. No entanto, ele introduz uma complexidade operacional significativa. O desempenho percebido não depende apenas do motor de consulta, mas da coordenação entre serviços, do estado do supervisor, da saúde do metadata store e da disponibilidade de brokers.

Em outras palavras, Druid oferece um estado da arte importante para workloads analíticos de eventos, mas o custo de atingir esse estado da arte não é trivial. Isso precisa aparecer no benchmark como métrica de custo operacional, cold start, transições de health e estabilidade do pipeline de ingestão. Um benchmark que medisse apenas latência de consulta e ignorasse a topologia multi-serviço de Druid estaria subestimando um aspecto central da tecnologia.

3.3.3 Druid no contexto de dashboards push-oriented

Para o caso deste projeto, Druid ocupa uma posição interessante. Ele consome o stream canônico do ledger via supervisor Kafka e serve o painel por meio de uma API dedicada. Em termos de convergência visual, o sistema se beneficia de snapshots autoritativos filtrados e de telemetria de pipeline, mas não opera como um mecanismo de incremental view maintenance fino. O gateway precisa coordenar o fluxo de eventos e os snapshots do backend para preservar coerência perceptual, especialmente quando filtros complexos entram em cena.

Isso significa que Druid é particularmente forte como backend de leitura analítica quente, mas menos natural como âncora incremental direta entre stream e frontend. Em cenários de alta cardinalidade ou de grande volume temporal, seu modelo de segmentos e planning pode continuar competitivo; em cenários de replay, bootstrap local e operação com recursos restritos, a topologia multi-serviço pode impor custos maiores que os observados em outras stacks.

3.3.4 Limites e riscos

Os principais riscos de Druid, para o caso estudado, residem na sensibilidade operacional do pipeline: supervisores em retry, dependências entre broker e router, estado do metadata store e custo de aquecimento da stack. Em uma publicação científica honesta, esses fatores não devem ser tratados como ruído contingente. Eles são parte do comportamento real da tecnologia em ambientes de engenharia. Assim, custo cognitivo e custo operacional local devem figurar como variáveis comparativas explícitas.

3.3.5 Bibliografia-base de Druid

- Yang, F. et al. Druid: A Real-Time Analytical Data Store.
- Documentação oficial do Apache Druid.
- Publicações técnicas do projeto sobre segment lifecycle, ingestion specs, roll-up, compaction e Kafka indexing service.

3.4 APACHE PINOT

3.4.1 Classe arquitetural

Apache Pinot foi concebido para serving OLAP de baixa latência em cenários com alto volume de consultas interativas, forte exigência de recorte dimensional e consumo contínuo de streams. Sua arquitetura distribui responsabilidades entre `Controller`, `Broker` e `Server`, combinando ingestão realtime com estratégias de indexação voltadas a consultas de baixa latência. No ecossistema industrial, Pinot se destacou sobretudo em casos

de analytics operacionais com alta taxa de leitura, grande fan-out de queries e necessidade de respostas quase imediatas para exploração por produto.

3.4.2 Internals relevantes

O desenho interno de Pinot combina armazenamento colunar, dicionários, indexes invertidos e, em alguns cenários, estruturas como `Star-Tree` para acelerar agregações recorrentes. Sua proposta não é apenas armazenar colunas, mas organizar o caminho entre consulta e dado por meio de estruturas que reduzam scanning desnecessário para workloads interativos. A ingestão realtime, baseada em consumo de streams como Kafka, torna Pinot particularmente atraente para modelos de leitura com atualização contínua.

No entanto, o que torna Pinot tecnicamente relevante para este benchmark é sua afinidade com modos `push-first`. Quando combinado a um frontend capaz de projetar localmente parte dos eventos `entry.created`, Pinot pode servir como backend de baixa latência para reancoragem periódica, enquanto a continuidade perceptual imediata é mantida no cliente. Trata-se de uma escolha arquitetural distinta da de Druid e da de Materialize: Pinot não internaliza a manutenção incremental completa do estado derivado, mas tampouco depende apenas de snapshots pesados para responder com rapidez.

3.4.3 Estado da literatura e fontes técnicas

Em comparação com Druid e ClickHouse, a literatura revisada por pares explicitamente centrada em Pinot é menos consolidada como corpus acadêmico canônico. Isso não invalida a tecnologia; apenas exige mais cuidado bibliográfico. Para uma escrita científica rigorosa, a base de Pinot deve combinar documentação oficial, textos de engenharia de produtores da tecnologia e, onde disponível, artigos técnicos sobre realtime OLAP, indexes e serving distribuído. Em termos epistêmicos, trata-se de uma área em que a literatura industrial de alta qualidade ainda ocupa parte do espaço que, em outros sistemas, foi mais formalizado por artigos clássicos.

3.4.4 Pinot no contexto do benchmark

No repositório, Pinot assume o papel de backend desacoplado com frontend especializado. Essa decisão é instrutiva: ela permite observar uma estratégia de convergência visual em que o cliente participa mais ativamente do processo, enquanto o backend fornece a âncora quente para consultas autoritativas. Em cenários de fila operacional, essa composição pode gerar excelente latência percebida. Em contrapartida, sua robustez de-

pende da qualidade da projeção local, da disciplina de reconciliação e da consistência entre eventos recebidos e snapshots reconsultados.

3.4.5 Limites e riscos

Os limites principais de Pinot, para este caso, emergem quando a complexidade do painel exige mais do que feed de eventos e resumos rápidos. Workspaces ricos em joins, catálogos e agregados contábeis cumulativos tendem a exigir maior coordenação entre frontend, API e backend. Portanto, o benchmark deve testar explicitamente estabilidade de `push-first`, divergência entre detalhamento e agregados e custo de ressincronização autoritativa.

3.4.6 Bibliografia-base de Pinot

- Documentação oficial do Apache Pinot.
- Textos técnicos da comunidade Pinot e de empresas mantenedoras como LinkedIn e StarTree.
- Literatura sobre indexes colunar-invertidos, `Star-Tree` e realtime OLAP em serving distribuído.

3.5 MATERIALIZE

3.5.1 Classe arquitetural

Materialize ocupa uma posição singular no corpus comparado. Ele não é, em sentido estrito, apenas um banco analítico colunar para serving quente, nem simplesmente um processador de stream desacoplado de interface SQL. Seu valor arquitetural reside justamente em articular semântica relacional declarativa com manutenção incremental de views sobre fontes streaming. Assim, Materialize se aproxima da categoria `incremental-streaming`, em que a definição do modelo de leitura participa do próprio mecanismo de convergência entre stream e estado derivado.

3.5.2 Fundamentos internos

O pano de fundo conceitual de Materialize está profundamente ligado a Timely Dataflow e Differential Dataflow. Em vez de recomputar resultados completos a cada mudança, o sistema propaga diferenças e frontier progress, permitindo atualização incremental de estruturas derivadas. Na prática, isso significa que `SOURCE`, `VIEW` e `MATERIALIZED VIEW` não são apenas objetos de catálogo; eles são partes de uma topologia incremental com semântica temporal e de progresso.

Essa característica é crucial para o caso deste benchmark. Em dashboards push-oriented, o que se deseja idealmente não é apenas consultar um estado quente rapidamente, mas aproximar o frontend de uma view derivada que se reconcilia continuamente com o stream canônico. Materialize oferece, ao menos em princípio, um encaixe natural para esse problema. Por isso sua trilha no repositório foi desenhada com bootstrap próprio, API dedicada, health incremental e gateway autoritativo por snapshot incremental.

3.5.3 Particularidades semânticas e custos cognitivos

O potencial de Materialize, contudo, não vem sem contrapartida. Sistemas incrementais impõem restrições semânticas reais. Funções não determinísticas, limites SQL associados ao modelo incremental, necessidade de atenção à monotonicidade e comportamento sob retrações não são detalhes cosméticos. Eles afetam diretamente o desenho das views e a tradução de modelos analíticos para o paradigma incremental. Em outras palavras, Materialize pode reduzir a distância entre stream e snapshot, mas cobra do engenheiro maior disciplina na modelagem relacional incremental.

Isso justifica a inclusão de `cognitive_cost` como métrica comparativa. Enquanto ClickHouse, Druid e Pinot frequentemente concentram a complexidade em serving, segmentação ou indexação, Materialize desloca parte do desafio para a expressão correta da topologia incremental. Em ambientes de benchmark sério, esse custo não deve ser escondido: ele é parte do valor explicativo do paradigma.

3.5.4 Materialize no caso de uso do repositório

No presente projeto, Materialize recebe diretamente `ledger-entries-v1`, bootstrapa conexão Kafka, source, view tipada e materialized views por domínio. A API `api_materialize` consulta essas views incrementais, enquanto o `realtime_gateway` fornece snapshots autoritativos progressivos ao frontend. Isso representa, de fato, uma configuração mais próxima do ideal conceitual do backend do que tratá-lo como mero SQL adapter. Nessa topologia, o frontend deixa de depender exclusivamente de projeção local por evento e passa a se ancorar em um snapshot incremental que vive mais próximo do fluxo.

Tal desenho é relevante para a literatura porque permite comparar, de forma concreta, um paradigma `hot-analytic` com um paradigma `incremental-streaming` sob o mesmo caso de uso funcional. Em vez de perguntar apenas qual banco responde consultas mais rápido, o benchmark pode perguntar qual arquitetura reduz melhor o tempo entre evento canônico e estado derivado utilizável por uma interface gerencial.

3.5.5 Limites e riscos

O principal risco ao avaliar Materialize é superestimar sua vantagem teórica sem medir seus custos reais. Alta cardinalidade, joins incrementais complexos, retrações e replay podem revelar tensões importantes entre elegância conceitual e custo operacional. Da mesma forma, um benchmark injusto pode subestimar a tecnologia se insistir em modelá-la com queries e contratos pensados para motores analíticos clássicos. A implicação metodológica é direta: Materialize deve ser avaliado em cenários desenhados para incremental view maintenance, sem deixar de ser comparado transversalmente em latência, corretude, recursos e robustez.

3.5.6 Bibliografia-base de Materialize

- Murray, D. G. et al. Naiad: A Timely Dataflow System.
- McSherry, F. et al. Differential Dataflow.
- Documentação oficial do Materialize.
- Publicações técnicas do projeto sobre progress tracking, sources, views materializadas, consistency fronts e streaming SQL.

3.6 SÍNTESE COMPARATIVA ENTRE CATEGORIAS

Ao final da análise, as quatro tecnologias distribuem-se em duas famílias comparativas principais. ClickHouse, Druid e Pinot são motores `hot-analytic`, ainda que com internals distintos. Eles tendem a operar melhor quando o read model consultável já foi suficientemente construído e quando o desafio dominante é responder bem sobre esse estado materializado. Materialize, por sua vez, aproxima-se de uma família `incremental-streaming`, na qual o custo e a qualidade da manutenção incremental do read model tornam-se parte integrante do sistema avaliado.

Essa distinção não implica que uma categoria substitua a outra. Ela implica apenas que o benchmark cientificamente correto deve reconhecer que as tecnologias respondem a perguntas parcialmente diferentes. O valor analítico do trabalho, portanto, está justamente em mostrar onde esses paradigmas convergem, onde divergem e sob quais métricas um parece mais adequado do que o outro para dashboards gerenciais alimentados por push.

3.7 CONCLUSÃO DO CAPÍTULO

O estado da arte tecnológico examinado neste capítulo revela que a comparação entre ClickHouse, Druid, Pinot e Materialize exige uma leitura em profundidade de internals, topologias operacionais e teorias implícitas de convergência entre evento e estado

consultável. Cada sistema encarna uma resposta diferente para o problema do serving analítico ou incremental. Por essa razão, o benchmark consolidado neste relatório não pode se contentar com uma tabela simples de latências. Ele precisa medir, com rigor, as propriedades que decorrem das escolhas internas de cada tecnologia: custo de bootstrap, regime de ingestão, comportamento sob filtros e replay, corretude contábil, estabilidade do tempo real e impacto sobre a experiência visual do frontend.

3.8 BIBLIOGRAFIA-BASE DO CAPÍTULO 3

- Abadi, D. J. et al. Column-stores versus row-stores: how different are they really?
- Chen, Y. et al. literatura técnica sobre serving distribuído e indexing para Pinot, complementada pela documentação oficial do projeto.
- ClickHouse, Inc. Documentação oficial e notas de engenharia sobre MergeTree, projections e materialized views.
- Druid committers and PMC. Documentação oficial do Apache Druid e material técnico sobre Kafka indexing service e segment lifecycle.
- Materialize, Inc. Documentação oficial e textos técnicos sobre streaming SQL, Timely Dataflow e Differential Dataflow.
- Stonebraker, M. et al. C-Store: A Column-oriented DBMS.
- Yang, F. et al. Druid: A Real-Time Analytical Data Store.

4 MODELO DO PROBLEMA E ARQUITETURA EXPERIMENTAL

4.1 DOMÍNIO FUNCIONAL

O caso de uso deste relatório é um ambiente de eventos contábil-financeiros sintéticos capaz de alimentar painéis executivos, contábeis e comerciais em tempo quase real. O domínio experimental inclui eventos de venda, frete, devolução, pagamento a fornecedor, transferências de tesouraria e pagamento de juros de capital de giro. Cada evento de domínio é traduzido em lançamentos contabilmente coerentes, respeitando a disciplina de dupla entrada.

O objetivo funcional não é apenas preservar um feed de eventos recentes, mas produzir três classes de leitura: um resumo executivo, um workspace detalhado e coleções filtráveis de entries e catálogos. Dessa forma, a stack experimental aproxima o benchmark de um contexto realista, em que o frontend depende simultaneamente de granularidade transacional, agregados consolidados e semântica contábil robusta.

4.2 CONTRATO CANÔNICO DOS DADOS

O contrato canônico do sistema é centrado no tópico Kafka `ledger-entries-v1`. O `storage_writer` executa a derivação contábil uma única vez e publica nesse stream os lançamentos normalizados. Essa escolha evita que cada backend precise reinterpretar o evento de negócio bruto de maneira autônoma, reduzindo o viés metodológico introduzido por pipelines de ingestão semanticamente distintos.

O contrato funcional compartilhado pelos frontends e APIs inclui, entre outros elementos:

1. `entry.created` como unidade de atualização transacional.
2. `dashboard.snapshot` como envelope autoritativo de reconciliação visual.
3. Endpoints de resumo, workspace, entries, filtros e master data.
4. Invariantes contábeis verificáveis a partir do mesmo ledger.

Essa combinação garante que a comparação entre backends ocorra sobre uma base funcional constante, mesmo quando a estratégia de serving varia.

4.3 ARQUITETURA DO REPOSITÓRIO

O repositório foi estruturado como monorepo operacional, contendo producer, OpenTelemetry Collector, Kafka, storage writer, APIs por backend, gateway de realtime, frontends especializados e scripts de benchmark. O fluxo macro pode ser descrito da seguinte forma:

1. O `producer` gera eventos sintéticos de negócio.
2. O `otel-collector` roteia a ingestão para o pipeline principal.
3. O `storage_writer` consolida a semântica contábil e publica o ledger canônico.
4. Cada backend consome o ledger por sua própria estratégia de ingestão.
5. APIs especializadas expõem o contrato de leitura ao frontend.
6. O `realtime_gateway` coordena stream e snapshots autoritativos para convergência visual.

Essa arquitetura é cientificamente relevante porque a unidade real de análise deixa de ser o banco isolado e passa a ser a composição entre banco, camada de consulta, gateway e frontend.

4.4 ESTRATÉGIAS DE CONVERGÊNCIA VISUAL POR BACKEND

Cada backend foi integrado ao mesmo problema funcional, mas com uma estratégia de convergência visual coerente com sua natureza interna:

1. ClickHouse: `authoritative resync`, com snapshots autoritativos combinados a eventos.
2. Druid: `filtered authoritative snapshot`, privilegiando snapshots consistentes sob filtros.
3. Pinot: `push-first`, com projeção local no frontend e reancoragem autoritativa.
4. Materialize: `authoritative incremental snapshot`, aproximando o frontend de views incrementais mantidas sobre o stream.

Essa diversidade permite comparar não apenas a performance do backend, mas a adequação de cada paradigma a formas distintas de alimentar o frontend sem degradar consistência visual.

4.5 OBSERVABILIDADE E PONTOS DE CONTROLE

O ambiente inclui endpoints de health e debug para master data, storage writer, realtime gateway e trilhas específicas de cada backend. Essa camada de observabilidade é

parte do desenho experimental, pois permite capturar transições de estado, bootstrap, degradação e readiness sem depender apenas de logs ad hoc. Em particular, a trilha Materialize expõe um endpoint de bootstrap incremental com métricas de hidratação, offsets Kafka e lag de views, o que a torna especialmente apropriada para avaliação de comportamento incremental real.

4.6 CONSEQUÊNCIAS METODOLÓGICAS

O modelo experimental adotado neste relatório possui duas implicações metodológicas principais. Primeiro, ele preserva um contrato funcional comum e um ledger canônico compartilhado. Segundo, ele admite que o modo de convergência visual é parte integrante do backend avaliado. Como consequência, o benchmark deixa de ser um teste simplificado de throughput e se torna uma comparação de arquiteturas de serving para dashboards gerenciais orientados por eventos.

5 METODOLOGIA DE BENCHMARK

5.1 TIPO DE PESQUISA

Esta pesquisa adota caráter experimental aplicado, com comparação controlada entre backends submetidos ao mesmo contrato funcional. O objetivo não é apenas obter números de latência, mas construir evidência empírica auditável sobre como diferentes paradigmas de serving se comportam sob um mesmo problema de negócio.

5.2 UNIDADE EXPERIMENTAL

Cada unidade experimental corresponde a uma rodada individualizada por backend e por cenário, executada em ambiente local controlado. O protocolo assume uma restrição pragmática importante: as stacks não devem concorrer simultaneamente pela mesma CPU, memória, disco e rede, pois isso contaminaria a comparação. Assim, a regra central é o isolamento por backend.

O identificador canônico de rodada adota o formato:

`<timestamp>__<backend>__<cenario>__run-<n>`

5.3 JANELA TEMPORAL DA RODADA

Na versão do protocolo efetivamente utilizada para a bateria conclusiva deste relatório, os tempos não foram tratados como literais soltos. Eles são materializados no código por constantes nomeadas e copiados para `metadata.json` sob a chave `phase_budget_seconds`, o que permite auditoria posterior da configuração aplicada em cada rodada.

Para as rodadas `report-conclusion`, executadas neste manuscrito, a parametrização observada foi a seguinte:

Parametro protocolar	Valor efetivo	Origem operacional
`collection_seconds`	270 s	argumento da rodada
`bootstrap_wait`	180 s	`BENCHMARK_DEFAULT_BOOTSTRAP_TI`
`backend_readiness_wait`	240 s	`BENCHMARK_DEFAULT_BACKEND_TIME`
`warmup`	10 s	`BENCHMARK_DEFAULT_WARMUP_SECON`
`per_probe_segment`	90 s	`max(collection_seconds / 3, BENCHMARK_MIN_SEGMENT_SECONDS)`

Parametro protocolar	Valor efetivo	Origem operacional
BENCHMARK_MIN_SEGMENT_SECONDS	45 s	piso operacional do coletor

Essa explicitude é metodologicamente relevante. Em vez de postular números arbitrários na escrita, o relatório passa a herdar os budgets reais do protocolo executado.

5.4 FLUXO OPERACIONAL DA COLETA

O protocolo de coleta segue cinco etapas.

5.4.1 Preparação

1. Executar `make stop ENV_FILE=<env>`.
2. Garantir ausencia de containers residuais.
3. Ativar exatamente um backend em `ACTIVE_STACKS`.
4. Registrar commit Git e configuracao efetiva de ambiente.

5.4.2 Bootstrap

1. Gerar um arquivo de ambiente derivado de `.env.example`, ativando exatamente um backend.
2. Resolver o conjunto de servicos por meio de `scripts/stack-selection.sh`.
3. Subir a stack via `docker compose --env-file <env> up --build -d ...`.
4. Executar `make populate ENV_FILE=<env>` para iniciar a geracao ativa de eventos e popular a trilha observada.
5. Aguardar estado minimo de saude para `master-data`, `storage-writer`, `realtime-gateway`, API do backend, frontend e endpoint de debug.

Essa variante foi necessária para preservar equivalência metodológica entre as stacks. O alvo genérico `make run` acoplava a coleta a um smoke test que, no caso de Druid, podia reprovar o sistema antes da materialização do datasource. A orquestração de benchmark foi, portanto, desacoplada do smoke genérico de desenvolvimento sem abrir mão de health checks explícitos.

5.4.3 Warm-up

1. Consumir endpoints de health, summary e workspace.
2. Confirmar que o frontend responde.
3. Confirmar que o websocket iniciou.

5.4.4 Coleta principal

1. Coletar latencias reais de API.
2. Coletar latencias reais de SQL nativo.
3. Coletar convergencia websocket e latencia percebida.
4. Coletar recursos computacionais dos containers.
5. Registrar timeline de health.

5.4.5 Encerramento

1. Capturar snapshots finais de debug, preservando inclusive falhas HTTP residuais como artefatos auditaveis quando algum endpoint de observabilidade nao responder com sucesso.
2. Consolidar os artefatos em JSON e CSV.
3. Derrubar a stack com `make stop`.

5.5 MÉTRICAS NUCLEARES

O benchmark combina seis famílias de métricas.

5.5.1 Bootstrap e disponibilidade

- `time_to_health_green_ms`
- `time_to_first_useful_snapshot_ms`
- `health_transition_count`
- `restart_count`

5.5.2 Latência de API

- p50, p95 e p99 de `summary`
- p50, p95 e p99 de `workspace`
- p50 e p95 de `entries`
- p50 e p95 de `filter-search`

5.5.3 Latência SQL nativa

- p50, p95 e p99 de consultas semanticamente equivalentes a resumo, workspace ou visões filtradas

5.5.4 Convergência visual no frontend

- `event_to_snapshot_visible_ms`

- `snapshot_rate_per_second`
- `entry_to_queue_visible_ms`
- `sale_to_sales_workspace_visible_ms`
- `frontend_time_to_first_meaningful_state_ms`

5.5.5 Corretude contábil

- `balance_sheet_difference`
- equivalência entre ativos e passivos mais patrimônio
- consistencia entre agregados e entries
- taxa de duplicidade indevida

5.5.6 Recursos computacionais

- `cpu_avg`
- `cpu_peak`
- `memory_avg_mb`
- `memory_peak_mb`
- tráfego de rede
- crescimento de disco

5.6 ARTEFATOS CANÔNICOS DA RODADA

Cada rodada deve produzir o seguinte conjunto mínimo de arquivos:

- `metadata.json`
- `api_latencies.json`
- `sql_latencies.json`
- `ws_metrics.json`
- `resources.json`
- `health_timeline.json`
- `debug_snapshots.json`
- `round_report.json`
- `summary.csv`

O arquivo `round_report.json` agrega os resultados brutos e derivados da rodada. O arquivo `summary.csv` sintetiza as colunas transversais necessárias para comparação estatística e composição de tabelas do manuscrito.

5.7 IMPLEMENTAÇÃO CONCRETA NO REPOSITÓRIO

O protocolo foi implementado por meio dos scripts localizados em `scripts/benchmark/`. O script `run_round.sh` orquestra a execução integral da rodada. Wrappers dedicados selecionam o backend. Scripts Python e Node coletam API, SQL, websocket, recursos, health e snapshots finais. Por fim, `consolidate_round_results.py` gera `round_report.json` e `summary.csv` no formato padrão. Como os budgets de fase são gravados em metadados, cada tabela do manuscrito pode ser confrontada diretamente com a configuração operada em tempo de execução.

Essa implementação é essencial para a validade da pesquisa, pois reduz a dependência de execução manual, melhora a reprodutibilidade e preserva rastreabilidade entre rodadas, configurações e commits.

5.8 AMEAÇAS À VALIDADE

Permanecem ameaças conhecidas à validade interna e externa: jitter do ambiente local, cache de disco, aquecimento assimétrico entre stacks, contenção residual de recursos e diferenças de custo operacional que não se refletem imediatamente em latência de consulta. Em vez de ocultar tais fatores, o protocolo os incorpora como parte do objeto de estudo, sobretudo por meio de timeline de health, artefatos de recursos e separação entre cold start e warm state.

5.9 FECHO METODOLÓGICO

A metodologia adotada transforma o benchmark em um experimento de sistema completo. O objeto observado não é apenas a consulta SQL, mas o ciclo que liga evento, ledger, backend, API, gateway e interface visual. Essa escolha é coerente com a pergunta científica do trabalho e sustenta a interpretação dos resultados em termos arquiteturais.

6 RESULTADOS COMPARATIVOS DA BATERIA CONCLUSIVA

6.1 OBJETIVO DESTA SEÇÃO

Esta seção apresenta a bateria comparativa efetivamente executada para os quatro backends investigados. Todos os números aqui reportados foram extraídos de rodadas reais do cenário `report-conclusion`, consolidadas em `summary.csv` e `round_report.json`, sem interpolação manual e sem substituição por valores exemplificativos.

6.2 CORPUS EMPÍRICO UTILIZADO

As rodadas comparadas compartilharam commit, host e duracao total de coleta, variando apenas o backend ativo.

Backend	Round ID	Commit	Duracao
ClickHouse	20260309T033332Z__clickhouse__report-conclusion__	221cabae82145de2805600377 3f83d8785f2705e	270 s
Druid	20260309T032706Z__druid__report-conclusion__	221cabae82145de2805600377 3f83d8785f2705e	270 s
Pinot	20260309T025603Z__pinot__report-conclusion__	221cabae82145de2805600377 3f83d8785f2705e	270 s
Materialize	20260309T030326Z__materialize__report-conclusion__	221cabae82145de2805600377 3f83d8785f2705e	270 s

Cada rodada produziu artefatos de API, SQL, websocket, health timeline, recursos computacionais e snapshots finais de debug. A tabela de comparação a seguir sintetiza apenas as métricas derivadas transversais mais diretamente comparáveis entre as quatro stacks.

6.3 MÉTRICAS CONSOLIDADAS COMPARATIVAS

Backend	API summary p95 (ms)	API workspace p95 (ms)	SQL entries_summary p95 (ms)	SQL summary_by_ingest p95 (ms)	First meaningful (ms)	Snapshots	Entries/s	Balance diff	Health transitions
ClickHouse	58.26	255.12	4.65	7.67	104	2.3369	78.3223	0.0	0
Druid	594.39	1748.37	55.85	60.19	45	2.3070	79.3577	0.0	1

Backend	API `summary` p95 (ms)	API `workspace` p95 (ms)	SQL `entries_count` p95 (ms)	SQL `summary_by_role` p95 (ms)	First meaningful (ms)	Snapshots/s	Entries/s	Balance diff	Health transitions
Pinot	89.07	198.71	3.04	3.32	42	0.0202	78.9367	0.0	1
Materialize	2143.40	1979.76	875.08	855.90	157	2.3353	53.4821	0.0	14

6.4 ANÁLISE POR DIMENSÃO OBSERVADA

6.4.1 Latência de API

No plano de serving HTTP, ClickHouse apresentou o melhor resultado para o endpoint `summary`, com p95 de 58.26 ms, seguido por Pinot com 89.07 ms. Para o endpoint `workspace`, Pinot obteve o menor p95 observado, 198.71 ms, com ClickHouse em seguida, 255.12 ms. Druid e Materialize exibiram custo substancialmente maior nesse contrato, com p95 de 1748.37 ms e 1979.76 ms, respectivamente.

Essa distribuição sugere que, neste ambiente e neste workload, as stacks orientadas a serving analítico direto preservam vantagem clara na superfície HTTP. A distância entre os dois melhores resultados e os dois piores é grande o suficiente para ser empiricamente relevante mesmo sem tratamento inferencial adicional.

6.4.2 Latência SQL nativa

No plano SQL, Pinot apresentou os menores p95 para as duas consultas sinteticas comparadas, 3.04 ms em `entries_count` e 3.32 ms em `summary_by_role`. ClickHouse manteve desempenho muito proximo, com 4.65 ms e 7.67 ms, respectivamente. Druid ocupou uma posicao intermediaria, na casa de dezenas de milissegundos. Materialize permaneceu claramente acima das demais stacks, com p95 de 875.08 ms e 855.90 ms.

Do ponto de vista estritamente consultivo, o resultado observado não favorece um paradigma incremental puro como primeira escolha para esse perfil de leitura. A vantagem de Pinot e ClickHouse, contudo, precisa ser interpretada em conjunto com as métricas de convergência visual e estabilidade, e não como argumento isolado de superioridade global.

6.4.3 Convergência visual e prontidão percebida

As métricas de frontend mostram uma dissociação importante entre custo de consulta e prontidão percebida. Pinot e Druid registraram os menores `frontend_time_to_first_meaningful`

42 ms e 45 ms, respectivamente. ClickHouse permaneceu em 104 ms e Materialize em 157 ms. Entretanto, essa leitura isolada seria enganosa se desconsiderasse `snapshot_rate_per_sec`

Nesse ponto, Druid, ClickHouse e Materialize mantiveram taxas semelhantes de snapshots, aproximadamente 2,3 por segundo. Pinot, por sua vez, exibiu taxa de apenas 0,0202 snapshots por segundo, apesar de excelente desempenho em API e SQL. Em termos metodológicos, esse contraste importa mais do que uma classificação simplista do menor tempo inicial: ele indica que a primeira percepção de prontidão pode ocorrer antes que a atualização autoritativa do painel atinja cadência comparável à das demais stacks.

6.4.4 Corretude contábil e estabilidade operacional

Em todas as rodadas observadas, `balance_sheet_difference` permaneceu em 0.0. Esse é um resultado central do experimento, pois elimina a hipótese de que ganhos aparentes de latência tenham sido comprados por perda de integridade contábil. Em outras palavras, a comparação se deu sob manutenção da corretude mínima do ledger agregado.

No plano de estabilidade, ClickHouse apresentou `health_transition_total` igual a 0, Druid e Pinot apresentaram 1 transição cada, e Materialize registrou 14 transições. A interpretação mais prudente é que a trilha Materialize permaneceu funcional, mas com maior sensibilidade operacional sob a carga observada no host local. Esse achado é coerente com sua pior posição nas métricas de latência de leitura e não deve ser separado da análise arquitetural.

6.5 SÍNTESE INTERPRETATIVA

Os dados coletados sustentam quatro afirmações empíricas.

Primeiro, não há dominância universal entre as stacks. ClickHouse liderou a latência de API para `summary`; Pinot liderou SQL e `workspace`; Druid combinou prontidão visual muito baixa com cadência de snapshots comparável à de ClickHouse; Materialize preservou integridade contábil e convergência visual razoável, mas ao custo de maior latência e maior instabilidade.

Segundo, a camada de experiência percebida não pode ser reduzida à latência SQL nativa. Druid ilustra isso de modo eloquente: embora seu `workspace` p95 tenha permanecido alto, 1748.37 ms, o frontend alcançou estado útil em 45 ms e manteve 2.307 snapshots por segundo. Materialize aponta na mesma direção, ainda que com custos mais severos. Isso reforça a tese de que, para dashboards near-real-time, o sistema de projeção e distribuição importa tanto quanto o mecanismo de consulta subjacente.

Terceiro, a taxa de snapshots do Pinot relativiza a excelência das suas leituras

HTTP e SQL. A stack foi extremamente rápida na superfície consultiva, mas o acoplamento entre evento e painel autoritativo mostrou cadência muito inferior nesta configuração observada. Portanto, qualquer conclusão favorável a Pinot deve ser explicitamente condicionada ao requisito dominante da aplicação.

Quarto, a corretude contábil foi preservada em todos os cenários observados. Esse fato melhora a qualidade epistêmica da comparação, pois impede que uma stack seja beneficiada por relaxamento semântico não declarado.

6.6 LIMITES DESTA EVIDÊNCIA

Esta seção consolida dados reais, mas não elimina as limitações do experimento. Cada backend foi representado por uma rodada conclusiva no mesmo host local, o que é suficiente para análise comparativa auditável, mas insuficiente para inferência estatística forte. Persistem ameaças conhecidas: jitter do WSL2, efeito de aquecimento residual, particularidades de configuração local de cada stack e ausência de repetições suficientes para estimar variância inter-rodada com robustez.

Em consequência, as conclusões a seguir devem ser lidas como afirmações empíricas circunscritas ao corpus coletado, e não como leis gerais sobre os produtos avaliados.

7 RESULTADOS COMPARATIVOS DERIVADOS DOS ARTEFATOS

Este capítulo é gerado automaticamente a partir dos artefatos em `artifacts/benchmark`. As tabelas abaixo não são mantidas manualmente: elas são reconstruídas no build do relatório, preservando rastreabilidade entre corpus empírico, tabelas e interpretação textual.

7.1 CORPUS, SELEÇÃO CANÔNICA E RASTREABILIDADE

A seleção canônica utiliza, para cada backend, a rodada mais recente do cenário `report-conclusio`
Rodadas auxiliares e tentativas substituídas permanecem documentadas para análise diagnóstica e para suporte a reruns de conformação.

Tabela 4 – Inventário completo das rodadas disponíveis no corpus de benchmark.

Classe	Backend	Cenário	Round ID	Início	Duração (s)	Erros	Transições	Dif. balanço
validação	materialize	validation-smoke	20260309T020307Z__materialize__validation-smoke__run-2	2026-03-09T02:03:07Z	90	48	5	0,00
validação	materialize	validation-sql	20260309T020746Z__materialize__validation-sql__run-3	2026-03-09T02:07:46Z	90	49	7	0,00
validação	materialize	validation-ready	20260309T021556Z__materialize__validation-ready__run-4	2026-03-09T02:15:56Z	90	0	4	0,00
substituída	clickhouse	report-conclusion	20260309T024926Z__clickhouse__report-conclusion__run-1	2026-03-09T02:49:26Z	270	99217	0	0,00
canônica	pinot	report-conclusion	20260309T025603Z__pinot__report-conclusion__run-1	2026-03-09T02:56:03Z	270	0	1	0,00
canônica	materialize	report-conclusion	20260309T030326Z__materialize__report-conclusion__run-1	2026-03-09T03:03:26Z	270	0	14	0,00
canônica	druid	report-conclusion	20260309T032706Z__druid__report-conclusion__run-1	2026-03-09T03:27:06Z	270	2385	1	0,00
canônica	clickhouse	report-conclusion	20260309T033332Z__clickhouse__report-conclusion__run-1	2026-03-09T03:33:32Z	270	0	0	0,00

Tabela 5 – Orçamento de fases das rodadas canônicas utilizadas na comparação principal.

Backend	Round ID	Bootstrap (s)	Readiness (s)	Warm-up (s)	Segmento (s)	Duração (s)
clickhouse	`20260309T033332Z__clickhouse__report-conclusion__run-1`	180	240	10	90	270
druid	`20260309T032706Z__druid__report-conclusion__run-1`	180	240	10	90	270
materialize	`20260309T030326Z__materialize__report-conclusion__run-1`	180	240	10	90	270
pinot	`20260309T025603Z__pinot__report-conclusion__run-1`	180	240	10	90	270

7.2 LATÊNCIA HTTP E SQL

Tabela 6 – Latência HTTP por endpoint nas rodadas canônicas.

Backend	Endpoint	Amostras	Sucesso	Erros	Média ms	p50 ms	p95 ms	p99 ms	Máx ms	Bytes médios
clickhouse	`health`	355	355	0	2,04	1,11	1,73	36,16	41,58	38,00
clickhouse	`summary`	355	355	0	33,27	26,94	58,26	69,35	109,56	48618,08
clickhouse	`workspace`	355	355	0	161,81	154,94	255,12	322,38	376,13	340973,17
clickhouse	`entries`	355	355	0	40,86	26,88	77,82	85,72	97,91	79332,68
clickhouse	`filter_search`	354	354	0	11,16	6,96	47,34	51,51	53,81	264,00
druid	`health`	54	54	0	84,08	64,04	129,56	969,61	1366,39	160,30
druid	`summary`	54	50	4	296,30	225,79	594,39	1001,78	1190,42	48906,90
druid	`workspace`	54	50	4	983,25	883,44	1748,37	2613,24	3099,00	343701,60
druid	`entries`	53	50	3	263,41	122,84	284,10	3064,67	5712,79	79605,76
druid	`filter_search`	53	53	0	1,76	1,42	3,45	6,79	8,85	77,00
materialize	`health`	15	15	0	744,45	972,36	1029,08	1044,73	1048,65	279,00
materialize	`summary`	15	15	0	1475,43	1087,59	2143,40	2144,74	2145,08	48852,53
materialize	`workspace`	15	15	0	1920,18	1948,21	1979,76	1997,42	2001,83	344905,60
materialize	`entries`	14	14	0	1097,86	1083,42	1245,35	1276,27	1284,00	79912,43
materialize	`filter_search`	14	14	0	899,21	906,26	1007,73	1028,55	1033,75	265,00
pinot	`health`	435	435	0	1,56	0,90	1,52	36,65	48,59	33,00
pinot	`summary`	435	435	0	41,73	26,33	89,07	102,76	1135,61	36139,39
pinot	`workspace`	435	435	0	125,27	109,93	198,71	288,32	568,79	256318,08
pinot	`entries`	434	434	0	32,78	18,50	86,16	101,12	157,12	58733,06
pinot	`filter_search`	434	434	0	2,46	1,39	2,13	41,91	46,84	77,00

Tabela 7 – Latência SQL por consulta sintética nas rodadas canônicas.

Backend	Consulta	Amostras	Sucesso	Erros	Linhas médias	Média ms	p50 ms	p95 ms	p99 ms	Máx
clickhouse	`entries_count`	6573	6573	0	1,00	3,22	2,82	4,65	10,29	61
clickhouse	`summary_by_role`	6573	6573	0	14,00	4,91	4,20	7,67	16,11	146
clickhouse	`filtered_entries`	6573	6573	0	50,00	5,54	4,88	8,60	17,34	61
druid	`entries_count`	2375	2375	0	1,00	12,12	7,67	55,85	71,93	100
druid	`summary_by_role`	2375	2375	0	14,00	16,16	11,14	60,19	75,67	100
druid	`filtered_entries`	2374	0	2374	N/D	N/D	N/D	N/D	N/D	N
materialize	`entries_count`	19	19	0	1,00	415,26	521,17	875,08	892,50	896
materialize	`summary_by_role`	19	19	0	14,00	385,83	191,97	855,90	1091,34	1150
materialize	`filtered_entries`	19	19	0	50,00	727,62	568,32	1432,35	1493,73	1509
pinot	`entries_count`	16090	16090	0	1,00	1,76	1,41	3,04	4,41	105
pinot	`summary_by_role`	16089	16089	0	14,00	1,94	1,58	3,32	4,79	86
pinot	`filtered_entries`	16089	16089	0	50,00	1,88	1,52	3,25	4,79	129

Tabela 8 – Tempos até o primeiro sucesso observável de API e SQL nas rodadas canônicas.

Backend	API summary	API workspace	API entries	Summary > 0	Workspace > 0	SQL count	SQL role	SQL filtere
clickhouse	51720	51756	51972	51720	51756	150579	150601	1506
druid	69834	71025	64029	69834	71025	152835	152870	N/
materialize	69146	70158	72062	69146	70158	167129	168088	1691
pinot	120916	122052	122360	151893	151982	219857	219888	2198

7.3 CONVERGÊNCIA VISUAL E EXPERIÊNCIA PERCEBIDA

Tabela 9 – Métricas de convergência visual e propagação de eventos nas rodadas canônicas.

Backend	Meaningful	1o snapshot	Entry -> fila	Evento -> snapshot	Venda -> sales	Snapshots/s	Entries/s	Snapsh
clickhouse	104	258	104	154	154	2,3369	78,3223	2
druid	45	556	45	511	508	2,3070	79,3577	2
materialize	157	1439	157	1282	524	2,3353	53,4821	2
pinot	42	42	155	N/D	N/D	0,0202	78,9367	

7.4 ESTABILIDADE OPERACIONAL E COBERTURA DIAGNÓSTICA

Tabela 10 – Prontidão observada pelos health checks durante as rodadas canônicas.

Backend	Endpoint	Amostras	Não ok	1o ok ms	Transições	Estados observados
clickhouse	`backend_debug`	134	134	N/D	0	running
clickhouse	`backend_health`	134	0	51730	0	ok
clickhouse	`master_data`	134	0	51691	0	ok
clickhouse	`realtime_gateway`	134	0	51728	0	ok
clickhouse	`storage_writer`	134	0	51722	0	ok
druid	`backend_debug`	122	122	N/D	0	desconhecido
druid	`backend_health`	122	3	64441	1	warming_up | ok
druid	`master_data`	122	0	53731	0	ok
druid	`realtime_gateway`	122	0	54489	0	ok
druid	`storage_writer`	122	0	53764	0	ok
materialize	`backend_debug`	53	53	N/D	14	ready | warming_up
materialize	`backend_health`	53	0	68156	0	ok
materialize	`master_data`	53	0	68124	0	ok
materialize	`realtime_gateway`	53	0	68153	0	ok
materialize	`storage_writer`	53	0	68149	0	ok
pinot	`backend_debug`	134	134	N/D	1	retrying | ready
pinot	`backend_health`	134	0	120928	0	ok
pinot	`master_data`	134	0	120878	0	ok
pinot	`realtime_gateway`	134	0	120924	0	ok
pinot	`storage_writer`	134	0	120915	0	ok

Tabela 11 – Cobertura dos snapshots finais de debug nas rodadas canônicas.

Backend	Snapshot	OK	Campos	Timestamp	Entries	Balance sheet
clickhouse	`backend_debug`	sim	10	não	não	não
clickhouse	`backend_entries`	sim	5	não	sim	não
clickhouse	`backend_filter_options`	sim	13	não	não	não
clickhouse	`backend_health`	sim	2	não	não	não
clickhouse	`backend_summary`	sim	7	sim	sim	sim
clickhouse	`backend_workspace`	sim	8	sim	sim	não
clickhouse	`master_data_health`	sim	2	não	não	não
clickhouse	`realtime_gateway_health`	sim	5	não	não	não
clickhouse	`storage_writer_health`	sim	2	não	não	não
druid	`backend_debug`	sim	6	não	não	não
druid	`backend_entries`	sim	5	não	sim	não
druid	`backend_filter_options`	não	0	não	não	não
druid	`backend_health`	sim	3	não	não	não
druid	`backend_summary`	sim	7	sim	sim	sim
druid	`backend_workspace`	sim	8	sim	sim	não
druid	`master_data_health`	sim	2	não	não	não
druid	`realtime_gateway_health`	sim	5	não	não	não
druid	`storage_writer_health`	sim	2	não	não	não
materialize	`backend_debug`	não	0	não	não	não
materialize	`backend_entries`	não	0	não	não	não
materialize	`backend_filter_options`	não	0	não	não	não
materialize	`backend_health`	não	0	não	não	não
materialize	`backend_summary`	não	0	não	não	não
materialize	`backend_workspace`	não	0	não	não	não
materialize	`master_data_health`	não	0	não	não	não
materialize	`realtime_gateway_health`	não	0	não	não	não
materialize	`storage_writer_health`	não	0	não	não	não
pinot	`backend_debug`	não	0	não	não	não
pinot	`backend_entries`	não	0	não	não	não
pinot	`backend_filter_options`	não	0	não	não	não
pinot	`backend_health`	não	0	não	não	não
pinot	`backend_summary`	não	0	não	não	não
pinot	`backend_workspace`	não	0	não	não	não
pinot	`master_data_health`	não	0	não	não	não
pinot	`realtime_gateway_health`	não	0	não	não	não
pinot	`storage_writer_health`	não	0	não	não	não

Tabela 12 – Falhas efetivamente observadas ao longo de todas as rodadas disponíveis.

Classe	Backend	Cenário	Round ID	Tipo	Probe	Erros	Amostras	Sucesso	p95 ms
validação	materialize	validation-smoke	20260309T020307Z__materialize__validation-smoke__run-2	sql	entries_count	16	16	0	N/D
validação	materialize	validation-smoke	20260309T020307Z__materialize__validation-smoke__run-2	sql	summary_by_role	16	16	0	N/D
validação	materialize	validation-smoke	20260309T020307Z__materialize__validation-smoke__run-2	sql	filtered_entries	16	16	0	N/D
validação	materialize	validation-sql	20260309T020746Z__materialize__validation-sql__run-3	sql	entries_count	17	17	0	N/D
validação	materialize	validation-sql	20260309T020746Z__materialize__validation-sql__run-3	sql	summary_by_role	16	16	0	N/D
validação	materialize	validation-sql	20260309T020746Z__materialize__validation-sql__run-3	sql	filtered_entries	16	16	0	N/D
substituída	clickhouse	report-conclusion	20260309T024926Z__clickhouse__report-conclusion__run-1	sql	entries_count	33073	33073	0	N/D
substituída	clickhouse	report-conclusion	20260309T024926Z__clickhouse__report-conclusion__run-1	sql	summary_by_role	33072	33072	0	N/D
substituída	clickhouse	report-conclusion	20260309T024926Z__clickhouse__report-conclusion__run-1	sql	filtered_entries	33072	33072	0	N/D
canônica	druid	report-conclusion	20260309T032706Z__druid__report-conclusion__run-1	api	summary	4	54	50	594,39
canônica	druid	report-conclusion	20260309T032706Z__druid__report-conclusion__run-1	api	workspace	4	54	50	1748,37
canônica	druid	report-conclusion	20260309T032706Z__druid__report-conclusion__run-1	api	entries	3	53	50	284,10
canônica	druid	report-conclusion	20260309T032706Z__druid__report-conclusion__run-1	sql	filtered_entries	2374	2374	0	N/D

7.5 RECURSOS COMPUTACIONAIS

Tabela 13 – Uso consolidado de recursos computacionais por backend canônico.

Backend	Contêineres	CPU média	CPU pico	Memória média MB	Memória pico MB	Maior contêiner MB
clickhouse	11	169,6943	183,6200	1923,79	2496,81	1339,39
druid	17	149,4246	101,1200	5281,33	5443,46	2046,98
materialize	11	109,3691	141,1800	2054,32	2255,34	1106,94
pinot	14	138,4581	116,7400	5144,59	5314,72	1499,14

A tabela consolidada de recursos permanece no corpo principal, mas o detalhamento por contêiner foi repartido por backend para preservar legibilidade sem truncar identificadores nem reduzir excessivamente o corpo tipográfico.

7.5.1 Backend Clickhouse

Tabela 14 – Detalhamento de recursos por papel de serviço para Clickhouse.

Papel	Contêiner	CPU média	CPU pico	Memória média MB	Memória pico MB	Amostras
`api`	`syntetic-ledger-dash-ui-api-1`	17,3189	51,2100	61,38	71,80	35
`engine:clickhouse`	`syntetic-ledger-dash-ui-clickhouse-1`	87,8651	183,6200	913,58	1339,39	35
`frontend`	`syntetic-ledger-dash-ui-frontend-1`	0,2937	2,7900	9,74	10,01	35
`kafka`	`syntetic-ledger-dash-ui-kafka-1`	28,1274	104,5100	423,76	515,60	35
`kafka-ui`	`syntetic-ledger-dash-ui-kafka-ui-1`	4,5617	51,4500	190,45	195,90	35
`master-data`	`syntetic-ledger-dash-ui-master-data-1`	1,8611	5,4800	42,89	43,31	35
`otel-collector`	`syntetic-ledger-dash-ui-otel-collector-1`	2,0783	2,8700	60,16	68,17	35
`producer`	`syntetic-ledger-dash-ui-producer-1`	3,7017	4,8800	26,80	28,97	35
`realtime-gateway`	`syntetic-ledger-dash-ui-realtime-gateway-1`	9,0069	25,4400	42,78	46,03	35
`storage-writer`	`syntetic-ledger-dash-ui-storage-writer-1`	5,5811	14,5000	55,16	55,73	35
`zookeeper`	`syntetic-ledger-dash-ui-zookeeper-1`	9,2983	80,8200	97,10	121,90	35

7.5.2 Backend Druid

Tabela 15 – Detalhamento de recursos por papel de serviço para Druid.

Papel	Contêiner	CPU média	CPU pico	Memória média MB	Memória pico MB	Amostras
`api-druid`	`syntetic-ledger-dash-ui-api-druid-1`	3,1917	11,9600	58,93	63,18	35
`druid-broker`	`syntetic-ledger-dash-ui-druid-broker-1`	46,6140	79,7700	2043,73	2046,98	35
`druid-coordinator`	`syntetic-ledger-dash-ui-druid-coordinator-1`	2,6523	48,6400	289,36	290,60	35
`druid-historical`	`syntetic-ledger-dash-ui-druid-historical-1`	0,4700	1,7000	326,09	326,20	35
`druid-middlemanager`	`syntetic-ledger-dash-ui-druid-middlemanager-1`	30,9046	100,6000	1003,71	1024,00	35
`druid-overlord`	`syntetic-ledger-dash-ui-druid-overlord-1`	0,7480	2,7300	297,46	298,40	35
`druid-postgres`	`syntetic-ledger-dash-ui-druid-postgres-1`	0,4851	5,0700	54,66	56,31	35
`druid-router`	`syntetic-ledger-dash-ui-druid-router-1`	10,7831	44,3900	252,09	255,90	35
`frontend-druid`	`syntetic-ledger-dash-ui-frontend-druid-1`	0,7731	1,4900	10,67	10,87	35
`kafka`	`syntetic-ledger-dash-ui-kafka-1`	25,2083	101,1200	424,71	511,90	35
`kafka-ui`	`syntetic-ledger-dash-ui-kafka-ui-1`	4,5320	51,8500	190,38	196,10	35
`master-data`	`syntetic-ledger-dash-ui-master-data-1`	0,6874	3,4300	41,71	41,93	35
`otel-collector`	`syntetic-ledger-dash-ui-otel-collector-1`	2,0389	3,4000	61,46	69,68	35
`producer`	`syntetic-ledger-dash-ui-producer-1`	3,4431	4,6400	26,40	28,60	35
`realtime-gateway`	`syntetic-ledger-dash-ui-realtime-gateway-1`	9,5943	21,2100	44,17	49,58	35
`storage-writer`	`syntetic-ledger-dash-ui-storage-writer-1`	1,5906	2,8200	50,61	51,23	35
`zookeeper`	`syntetic-ledger-dash-ui-zookeeper-1`	5,7080	84,0900	105,18	122,00	35

7.5.3 Backend Materialize

Tabela 16 – Detalhamento de recursos por papel de serviço para Materialize.

Papel	Contêiner	CPU média	CPU pico	Memória média MB	Memória pico MB	Amostras
`api-materialize`	`syntetic-ledger-dash-ui-api-materialize-1`	12,7200	56,4800	58,99	75,79	35
`frontend-materialize`	`syntetic-ledger-dash-ui-frontend-materialize-1`	0,1651	0,6700	10,31	10,43	35
`kafka`	`syntetic-ledger-dash-ui-kafka-1`	26,5540	102,5300	427,45	508,70	35
`kafka-ui`	`syntetic-ledger-dash-ui-kafka-ui-1`	0,1983	2,0900	218,83	219,10	35
`master-data`	`syntetic-ledger-dash-ui-master-data-1`	0,3306	4,6800	43,59	43,78	35
`materialized`	`syntetic-ledger-dash-ui-materialized-1`	47,0003	112,1000	1018,09	1106,94	35
`otel-collector`	`syntetic-ledger-dash-ui-otel-collector-1`	2,3309	5,8600	58,82	66,78	35
`producer`	`syntetic-ledger-dash-ui-producer-1`	3,6677	6,3600	33,33	35,69	35
`realtime-gateway`	`syntetic-ledger-dash-ui-realtime-gateway-1`	6,1231	13,2600	42,89	45,61	35
`storage-writer`	`syntetic-ledger-dash-ui-storage-writer-1`	1,5969	2,2400	47,50	47,91	35
`zookeeper`	`syntetic-ledger-dash-ui-zookeeper-1`	8,6823	141,1800	94,51	94,61	35

7.5.4 Backend Pinot

Tabela 17 – Detalhamento de recursos por papel de serviço para Pinot.

Papel	Contêiner	CPU média	CPU pico	Memória média MB	Memória pico MB	Amostras
`api-pinot`	`syntetic-ledger-dash-ui-api-pinot-1`	14,0256	50,8900	60,66	66,57	36
`frontend-pinot`	`syntetic-ledger-dash-ui-frontend-pinot-1`	0,7744	1,7100	9,76	9,85	36
`kafka`	`syntetic-ledger-dash-ui-kafka-1`	27,6675	101,0300	420,00	489,20	36
`kafka-ui`	`syntetic-ledger-dash-ui-kafka-ui-1`	0,1639	2,9100	185,38	186,00	36
`master-data`	`syntetic-ledger-dash-ui-master-data-1`	2,0878	8,6000	39,35	39,62	36
`otel-collector`	`syntetic-ledger-dash-ui-otel-collector-1`	2,0558	3,3100	55,37	67,69	36
`pinot-broker`	`syntetic-ledger-dash-ui-pinot-broker-1`	43,5428	102,7600	1276,22	1280,00	36
`pinot-controller`	`syntetic-ledger-dash-ui-pinot-controller-1`	1,6181	15,6800	1270,56	1274,88	36
`pinot-server`	`syntetic-ledger-dash-ui-pinot-server-1`	20,9681	95,4800	1485,48	1499,14	36
`pinot-zookeeper`	`syntetic-ledger-dash-ui-pinot-zookeeper-1`	6,1014	116,7400	119,47	147,10	36
`producer`	`syntetic-ledger-dash-ui-producer-1`	3,4728	4,2800	26,85	28,93	36
`realtime-gateway`	`syntetic-ledger-dash-ui-realtime-gateway-1`	6,8264	9,5100	42,98	44,45	36
`storage-writer`	`syntetic-ledger-dash-ui-storage-writer-1`	1,6053	2,0600	53,59	54,59	36
`zookeeper`	`syntetic-ledger-dash-ui-zookeeper-1`	7,5483	66,8300	98,93	126,70	36

7.6 SÍNTESE ORIENTADA PELOS DADOS

As tabelas confirmam que o corpus canônico final preserva corretude contábil total em todos os backends, mas não preserva equivalência operacional entre eles. ClickHouse e Pinot dominam as superfícies HTTP e SQL na maior parte das consultas observadas, enquanto Druid e Materialize exibem custos muito superiores nessas mesmas interfaces.

Ao mesmo tempo, a camada de convergência visual exige leitura menos simplista. Druid mantém `frontend_time_to_first_meaningful_state_ms` muito baixo e cadência de snapshots comparável à de ClickHouse, mesmo com warm-up inicial e erros transitórios na superfície HTTP. Pinot, por contraste, é extremamente competitivo em leitura direta, mas sua cadência autoritativa de snapshots permanece muito inferior nas métricas coletadas.

O inventário histórico das rodadas também mostra por que o artigo não deve se apoiar apenas na última tabela-resumo: houve tentativas substituídas e cenários de validação com falhas concentradas em probes SQL e em fases de readiness. Isso justifica separar explicitamente corpus canônico, corpus auxiliar e evidência diagnóstica, além de orientar futuros reruns de dupla conformação nas trilhas com maior incidência de erro.

8 ANÁLISE TÉCNICA POR BACKEND

8.1 PROPÓSITO DESTE CAPÍTULO

O capítulo anterior consolidou a leitura comparativa direta das métricas mais visíveis do corpus. Essa visão é necessária, mas ainda insuficiente para explicar por que cada stack se comportou como se comportou. Um benchmark de sistema completo não produz apenas uma ordenação de vencedores e perdedores; ele produz também um mapa causal provisório entre arquitetura interna, topologia operacional, superfície de consulta, mecanismo de convergência visual e custo de sustentação. É precisamente esse mapa que este capítulo procura construir.

Em vez de tratar ClickHouse, Druid, Pinot e Materialize como caixas pretas que recebem eventos e devolvem tempos de resposta, a análise a seguir reconstrói, backend a backend, quais propriedades técnicas parecem ter moldado o comportamento observado. A pergunta orientadora deixa de ser "quem foi mais rápido?" e passa a ser "qual combinação entre internals, integração no repositório e regime do experimento explica os resultados medidos?". Essa mudança é essencial para evitar conclusões superficiais. Um backend pode ser rápido porque opera sobre um estado já bem aquecido; outro pode parecer mais lento porque absorve internamente responsabilidades que, noutra stack, foram deslocadas para a aplicação, para o gateway ou para o frontend.

Há ainda uma segunda razão para esta leitura aprofundada. O corpus deste relatório não registra apenas quatro rodadas finais. Ele preserva também tentativas substituídas, rodadas de validação e evidências diagnósticas que mostram warm-up irregular, falhas transitórias, indisponibilidade parcial de endpoints de debug e assimetrias de custo operacional. Essas trilhas não devem ser tratadas como resíduos descartáveis. Elas são parte da realidade do sistema e ajudam a interpretar por que certos resultados finais são mais estáveis, mais frágeis ou mais caros de reproduzir.

8.2 CLICKHOUSE SOB O PROTOCOLO DO EXPERIMENTO

8.2.1 Estrutura funcional da trilha ClickHouse

Na configuração deste repositório, ClickHouse aparece como a materialização mais clássica do paradigma hot-analytic. O ledger canônico já chega semanticamente depurado pelo storage writer, que realiza a derivação contábil, preserva a disciplina de dupla entrada e expõe ao backend um fluxo normalizado de entries. Isso significa que a trilha ClickHouse não precisa resolver o problema conceitual mais duro do domínio, que é

a tradução de eventos de negócio heterogêneos em um modelo financeiro consistente. Em vez disso, ela recebe um stream já pronto para ser indexado e servido.

Esse detalhe metodológico é decisivo. Ele permite que o benchmark observe ClickHouse em um regime particularmente favorável à sua natureza: ingestão relativamente simples, serving consultivo intenso e necessidade de responder bem a resumos, workspaces e consultas filtradas sobre um conjunto de dados em crescimento. A força de ClickHouse, nesse contexto, não vem de uma semântica incremental sofisticada, mas da combinação entre armazenamento colunar, compressão, pruning e custo baixo para agregações recorrentes quando a ordenação e a modelagem tabular estão ajustadas ao workload.

8.2.2 Superfície HTTP observada

A rodada canônica final de ClickHouse apresentou o melhor p95 de `summary` na API entre as stacks comparadas. Esse resultado é coerente com a função que o backend desempenha no experimento. O endpoint `summary` representa precisamente o tipo de consulta que motores colunares bem organizados tendem a servir com alta eficiência: agregações consolidadas, cardinalidade moderada e conjunto de colunas relativamente estável. O mesmo raciocínio ajuda a explicar o bom desempenho em `entries`, ainda que a diferença para Pinot seja menor nessa superfície.

Já o resultado de `workspace` é levemente distinto. ClickHouse não foi o melhor, embora tenha permanecido em faixa bastante competitiva. Isso sugere que o workspace, por sua própria natureza, custa mais do que o resumo executivo simples. Nessa API, o sistema precisa combinar mais dados, transportar payload significativamente maior e preservar coerência entre subconjuntos heterogêneos do dashboard. O fato de ClickHouse continuar bem posicionado mesmo assim reforça a interpretação de que a trilha estava bem calibrada para leitura quente de estado já materializado.

8.2.3 SQL nativo e leitura quente

A leitura SQL de ClickHouse permaneceu em segundo lugar, atrás apenas de Pinot, em todas as consultas canônicas bem-sucedidas. Essa proximidade é importante porque revela que a superioridade consultiva de Pinot não implica uma derrota estrutural de ClickHouse; ela mostra apenas que, neste corpus específico, Pinot extraiu ainda mais eficiência em certas consultas sintéticas de baixa cardinalidade. Para fins de engenharia, isso significa que a decisão entre os dois motores não pode ser baseada apenas em uma diferença pontual de milissegundos. É preciso considerar qual dos dois exige menos coordenação periférica, menos ajuste fino, menos custo operacional local e menos complexidade para preservar a experiência percebida do painel.

8.2.4 Convergência visual e relação com o gateway

O dado mais instrutivo da trilha ClickHouse talvez não seja sua latência pura, mas o fato de combinar boa superfície consultiva com uma cadência autoritativa de snapshots muito superior à observada em Pinot. Esse ponto importa porque demonstra que o sucesso de ClickHouse, no experimento, não se limitou ao banco isolado. Houve também uma boa acomodação entre banco, API, gateway e frontend. Em outras palavras, a stack não ficou refém de uma escolha entre duas virtudes incompatíveis. Ela conseguiu preservar boa leitura HTTP e SQL sem sacrificar o ritmo de reconvergência autoritativa do painel.

Isso sugere que a estratégia híbrida associada a ClickHouse, baseada em feed de eventos para continuidade imediata e snapshots periódicos para reconciliação, está suficientemente madura para o workload testado. Não significa que ela resolveria qualquer cenário. Significa apenas que, neste domínio e neste protocolo, a coordenação extra exigida por um motor hot-analytic não produziu penalidade decisiva na experiência percebida.

8.2.5 Diagnóstico da tentativa substituída

O ponto que impede uma leitura triunfalista da stack é a existência de uma rodada anterior substituída com um volume massivo de falhas SQL. Essa tentativa mostra que a trilha ClickHouse não deve ser lida como naturalmente robusta em qualquer configuração. O experimento preservou evidência de que certas combinações de estado, bootstrap, carga ou preparação do ambiente foram capazes de degradar severamente a coleta SQL. É exatamente por isso que a rodada final precisa ser interpretada não apenas como "o resultado do backend", mas como "o resultado do backend quando a trilha operacional correta já havia sido estabilizada".

Em termos científicos, isso não invalida o bom resultado final. Mas impõe cautela: o benchmark não autoriza concluir que ClickHouse é simplesmente rápido; ele autoriza concluir que ClickHouse foi rápido na rodada canônica estabilizada, ao passo que uma tentativa anterior mostrou sensibilidade importante à preparação experimental. Esse contraste torna a stack mais interessante, e não menos. Ele mostra que performance observada e robustez de repetição não são a mesma coisa.

8.3 DRUID SOB O PROTOCOLO DO EXPERIMENTO

8.3.1 A complexidade da topologia Druid no corpus

Nenhuma outra stack do corpus deixa tão claro quanto Druid que benchmarking de backend é benchmarking de sistema multi-serviço. O que se mede ali não é apenas um

mecanismo de consulta. Mede-se também a coordenação entre broker, router, supervisor de ingestão, segmentos, metadata store e readiness do ecossistema inteiro. Esse traço distingue Druid não apenas de Materialize, mas também de ClickHouse e Pinot, cujas integrações no repositório, embora complexas, não expõem com a mesma nitidez uma coreografia operacional tão distribuída.

Essa característica ajuda a explicar uma aparente contradição do corpus: Druid teve p95 altos em `summary` e `workspace`, mas ao mesmo tempo apresentou uma das melhores métricas de `frontend_time_to_first_meaningful_state_ms` e manteve taxa de snapshots muito próxima da de ClickHouse. Em leitura simplista, isso pareceria incoerente. Em leitura arquitetural, faz sentido. A prontidão percebida do frontend não depende apenas do tempo de cada consulta pesada; depende também de quando o sistema passa a fornecer material suficiente para que o painel entre em estado semanticamente útil e continue se reconciliando com o fluxo.

8.3.2 Warm-up e transição de readiness

Os próprios artefatos deixam claro que a trilha Druid passou por warming up real. Os primeiros 503 em endpoints de API e as transições registradas no health timeline não são ruído desprezível. Eles demonstram que a stack não entra imediatamente em regime estável no mesmo modo que uma topologia mais compacta. Essa latência de aquecimento tem implicação dupla. De um lado, eleva o custo operacional local do backend. De outro, relativiza qualquer comparação em que se observem apenas os instantes posteriores ao aquecimento.

No entanto, há uma leitura complementar importante. O fato de Druid ter se estabilizado e, uma vez estabilizado, ter sustentado cadência de snapshots e prontidão percebida competitivas, indica que o custo adicional de arranque não o impede de ser interessante para dashboards push-oriented. Ele apenas desloca parte do problema para a engenharia operacional. Isso é coerente com a literatura prática da tecnologia: Druid frequentemente recompensa a organização correta da ingestão e do serving, mas cobra mais disciplina de operação para chegar a esse ponto.

8.3.3 A anomalia de `filtered_entries`

O maior ponto de atenção na rodada canônica final de Druid é o fracasso sistemático da consulta `filtered_entries` no corpus SQL. Esse dado não pode ser tratado como detalhe menor, porque ele rompe a comparabilidade plena entre as superfícies SQL canônicas. Ao mesmo tempo, seria um erro transformá-lo em condenação total da tecnologia. O mais defensável é tratá-lo como uma interseção entre semântica da consulta, estado da ingestão, tradução SQL e sensibilidade da trilha específica construída no repositório.

Para o artigo, a consequência é metodológica. Em vez de esconder a falha e manter apenas a tabela resumida, é preciso explicitar que a bateria conclusiva de Druid ficou parcialmente incompleta na superfície SQL filtrada, apesar de ter permanecido funcional e até competitiva no plano de convergência visual. Isso reforça a ideia central do trabalho: não existe uma única dimensão dominante capaz de absorver as demais.

8.3.4 Economia de experiência percebida

Druid mostrou que, em certos cenários, o usuário pode receber uma experiência inicial muito boa antes que as superfícies consultivas mais pesadas atinjam sua melhor forma. Em aplicações gerenciais, isso não é trivial. A utilidade prática do painel muitas vezes depende de o operador voltar a ver um estado coerente rapidamente, ainda que parte das consultas profundas ou parte dos filtros sofisticados exija mais tempo para convergir. Em outras palavras, a stack sugere uma forma de economia arquitetural na qual a experiência inicial pode ser muito boa mesmo quando a latência total de consulta não é líder.

8.3.5 Leitura crítica da trilha Druid

O backend sai do corpus com um perfil ambíguo, porém intelectualmente valioso. Ele não vence na leitura crua de p95 HTTP nem oferece a superfície SQL mais limpa. Ainda assim, aparece como uma solução plausível para cenários em que o problema dominante seja manter ritmo autoritativo de snapshots e boa percepção inicial do frontend, desde que a equipe aceite a maior exigência operacional e lide explicitamente com a fragilidade de certos caminhos de consulta.

8.4 PINOT SOB O PROTOCOLO DO EXPERIMENTO

8.4.1 Superioridade consultiva e seus limites

Pinot foi o melhor backend do corpus em consultas SQL e o melhor em parte da superfície HTTP. Isso já seria suficiente para destacá-lo. Mas o dado mais interessante é que essa superioridade apareceu de forma consistente em várias consultas de natureza distinta, o que sugere um casamento particularmente eficiente entre o workload sintético do relatório e o estilo de serving operacional para o qual Pinot foi desenhado.

Ao mesmo tempo, o próprio corpus impede que essa superioridade seja convertida em vitória universal. O backend exibiu taxa autoritativa de snapshots dramaticamente inferior à das demais stacks. Portanto, a leitura correta não é “Pinot é o melhor”; a leitura correta é “Pinot foi o melhor motor de leitura direta no protocolo observado,

mas essa vantagem não se converteu automaticamente em cadência autoritativa equivalente na camada de atualização visual".

8.4.2 O papel do frontend especializado

Essa dissociação é coerente com a integração adotada no repositório. A trilha Pinot foi desenhada de modo particularmente favorável a um frontend mais ativo na projeção local de eventos. Isso significa que parte da agilidade percebida não depende apenas do backend responder rápido, mas também da disciplina com que o cliente mantém estado transitório útil antes da reconciliação completa. Em termos de arquitetura de produto, isso é excelente quando a prioridade máxima é fluidez operacional imediata. Em termos de governança técnica, porém, desloca responsabilidade para a borda.

Uma consequência direta dessa escolha é que a stack precisa ser julgada não apenas por sua leitura quente, mas pela distância entre a performance consultiva e a atualização autoritativa efetivamente medida. Se essa distância se torna grande demais, há risco de que o sistema pareça responsivo, mas permaneça dependente de uma negociação mais complexa entre evento local, reancoragem periódica e completude do dashboard.

8.4.3 Ausência de cobertura diagnóstica no fim da rodada

Outro ponto crítico da trilha Pinot é a cobertura final reduzida nos snapshots de debug, que no corpus aparece mais limitada do que a observada em ClickHouse e Druid. Isso não destrói o resultado consultivo, mas empobrece a auditabilidade pós-rodada. Para um artigo técnico aprofundado, esse detalhe importa bastante, porque parte da força do benchmark está precisamente em preservar rastros que expliquem o que aconteceu quando algo vai bem e quando algo vai mal. Um backend muito rápido, porém menos auditável ao final da rodada, produz uma forma diferente de custo de engenharia.

8.4.4 Leitura arquitetural final de Pinot

O que Pinot sugere, então, é uma solução particularmente forte quando a organização está disposta a aceitar um frontend mais participante e deseja maximizar a velocidade de serving sobre um estado analítico quente. A questão crítica passa a ser governar corretamente a diferença entre rapidez consultiva e ritmo de ressincronização autoritativa. Se a aplicação tolera ou explora bem essa arquitetura, Pinot torna-se extremamente atraente. Se a aplicação exige atualização autoritativa muito frequente do painel integral, a vantagem consultiva pode não bastar.

8.5 MATERIALIZE SOB O PROTOCOLO DO EXPERIMENTO

8.5.1 O backend mais teoricamente promissor do corpus

Materialize ocupa no relatório a posição mais delicada e mais interessante. Em tese, é o backend cujo paradigma mais naturalmente se aproxima da ambição conceitual do problema: reduzir a distância entre stream canônico e estado derivado consultável por meio de manutenção incremental. Em outras palavras, ele promete resolver, no coração do próprio sistema, parte do problema que as demais stacks resolvem por composição entre writer, API, gateway e frontend.

Essa promessa, porém, só tem valor científico se for testada contra custos reais. E o corpus mostra custos reais expressivos. Materialize foi a stack mais cara em várias consultas HTTP e SQL, além de registrar a maior volatilidade operacional no health timeline. Portanto, sua relevância no trabalho não decorre de ser a "vencedora prática", mas de funcionar como caso-limite epistemicamente rico: a tecnologia mais alinhada ao ideal incremental não foi, neste protocolo, a mais eficiente como superfície de leitura.

8.5.2 Dissociação entre utilidade inicial e custo consultivo

O dado mais esclarecedor de Materialize talvez seja exatamente a dissociação entre utilidade inicial e custo consultivo. O frontend alcança estado significativo antes que as consultas pareçam baratas. Isso aponta para uma propriedade arquitetural importante: a experiência do usuário não está rigidamente acoplada ao tempo de cada consulta isolada. A stack consegue, ao menos parcialmente, transformar atualização incremental e snapshot autoritativo progressivo em utilidade operacional antes que a superfície consultiva integral se torne comparável à de motores hot-analytic.

Em termos de teoria de sistemas interativos, isso é um resultado relevante. Mostra que latência de query e latência percebida pertencem a camadas relacionadas, mas não idênticas. Em termos de engenharia, contudo, o resultado só vale se o custo operacional e a instabilidade não anularem a viabilidade da solução em produção.

8.5.3 Volatilidade e cobertura de debug

As transições de health e a baixa cobertura de snapshots finais de debug em Materialize não podem ser tratadas como pequenos acidentes. Elas sugerem que a trilha incremental, do modo como foi instrumentada no repositório, ainda não estava plenamente madura no momento da coleta. Isso não prova que a tecnologia seja intrinsecamente instável. Prova apenas que a configuração concreta usada no corpus ainda impunha fricção suficiente para aparecer nas métricas de operação.

Esse é um ponto especialmente importante para o artigo, porque ajuda a escapar

de uma leitura binária. Não seria correto dizer que Materialize "falhou", pois preservou corretude contábil e entregou indícios fortes de valor arquitetural. Também não seria correto dizer que "venceu conceitualmente", ignorando a realidade das consultas e da operação. O resultado mais honesto é que Materialize funcionou como a stack de maior interesse analítico e de menor maturidade prática relativa no corpus medido.

8.5.4 O valor científico da trilha Materialize

Mesmo sem liderar as métricas de leitura, Materialize talvez seja a tecnologia que mais obriga o relatório a elevar o nível do debate. Ele impede que a avaliação de backends seja reduzida a um ranking de p95. Ao fazer isso, força a pesquisa a reconhecer que o problema do trabalho não é simplesmente acelerar consultas, mas articular semântica incremental, experiência percebida, corretude financeira e custo operacional. Esse deslocamento conceitual é uma das contribuições mais importantes do corpus.

8.6 SÍNTESE TÉCNICA DO CAPÍTULO

Analisadas isoladamente, as stacks podem parecer apenas alternativas de implementação para a mesma função. Analisadas à luz do experimento, revelam-se arranjos com lógicas internas bem diferentes. ClickHouse destacou-se como a solução mais consistente entre leitura rápida e boa cadência autoritativa após estabilização. Druid apareceu como o backend em que o custo operacional do arranque pesa mais, mas onde a experiência percebida e a cadência de snapshots podem permanecer muito boas quando o sistema entra em regime. Pinot confirmou superioridade consultiva robusta, ao custo de depender mais de arquitetura `push-first` e menos de reancoragem frequente do estado global. Materialize, por fim, mostrou o maior alinhamento conceitual com o ideal incremental do problema, mas também o maior custo para tornar esse ideal operacionalmente estável.

O valor do benchmark, portanto, não está em decretar um vencedor único. Está em mostrar que cada backend oferece uma solução distinta para um mesmo sistema de referência e que essas soluções cobram preços diferentes em consulta, convergência, operação e auditabilidade. Esse é o tipo de resultado que um artigo técnico amplo precisa registrar, porque ele continua relevante mesmo quando as posições relativas mudarem em futuras rodadas de rerun.

9 DISCUSSÃO COMPARATIVA AMPLIADA

9.1 POR QUE A TABELA-RESUMO NÃO BASTA

Em benchmark de banco de dados, é comum que a discussão final se resuma a uma tabela curta de tempos e a uma conclusão do tipo "o sistema X foi mais rápido". Esse formato pode ser útil para workloads homogêneos, com uma única superfície de consulta e poucas exigências semânticas. Não é o caso deste relatório. O objeto de análise não é um conjunto de queries artificialmente isoladas; é a cadeia que conecta evento de negócio, ledger, backend, API, gateway realtime e frontend gerencial. Quando o objeto é uma cadeia e não uma única função, o sentido de "melhor" muda radicalmente.

O corpus final deixa isso evidente. Se a análise se limitasse ao SQL, Pinot seria o vencedor claro. Se a leitura se limitasse ao endpoint `summary`, ClickHouse teria vantagem mais forte. Se o foco recaisse sobre o tempo até o primeiro estado útil do frontend, Druid e Pinot pareceriam os mais atraentes. Se a ênfase estivesse na proximidade entre ideal incremental e modelo conceitual do problema, Materialize ganharia centralidade analítica. Cada recorte produz uma resposta distinta. O papel deste capítulo é organizar esses recortes em uma discussão coerente, em vez de escolher arbitrariamente apenas um deles.

9.2 DIMENSÃO 1: LATÊNCIA DE CONSULTA NÃO É LATÊNCIA DE SISTEMA

A primeira tese da discussão ampliada é que latência de consulta e latência de sistema não são equivalentes. A literatura de bancos frequentemente privilegia métricas ligadas ao tempo de execução de query, throughput e escalabilidade. Tais medidas continuam importantes aqui, mas o corpus mostra que elas não esgotam o problema prático dos dashboards gerenciais. Em um painel orientado a eventos, o usuário não interage com um benchmark SQL; ele interage com um estado visual em processo de convergência.

Isso explica por que Druid pode parecer relativamente caro em `workspace` e ainda assim entregar prontidão visual muito competitiva. Explica também por que Materialize, mesmo perdendo em várias consultas, continua relevante quando o problema é encurtar a distância entre mudança no stream e utilidade operacional do snapshot. A lição não é que consulta deixou de importar. A lição é que consulta importa como parte de um sistema maior, não como soberana absoluta da decisão arquitetural.

9.3 DIMENSÃO 2: ESTADO QUENTE VERSUS ESTADO INCREMENTAL

A segunda tese diz respeito à diferença entre servir um estado já quente e manter incrementalmente um estado derivado. Os resultados deste relatório sugerem que servir estado quente continua sendo uma estratégia extremamente poderosa. ClickHouse e Pinot mostram isso com clareza. Quando o modelo de leitura está materializado e a API sabe explorá-lo bem, a velocidade consultiva pode ser excepcional.

Entretanto, o corpus também sugere que essa vantagem não encerra a discussão. O que o paradigma hot-analytic ganha em consulta ele pode perder em distância conceitual para o problema do dashboard autoritativo. Nesses sistemas, parte do trabalho de manter coerência entre fluxo, snapshot e frontend desloca-se para a borda da aplicação. O que Materialize torna visível é exatamente o outro lado do espelho: internalizar mais da lógica incremental pode aproximar o backend do problema semântico, ainda que isso venha acompanhado de custos operacionais e de consulta que, no estado atual da trilha, permanecem elevados.

Em outras palavras, o benchmark mostra uma tensão estrutural entre duas ambições. A primeira é responder rapidamente sobre um estado quente. A segunda é reduzir ao máximo o custo cognitivo e temporal entre o stream e o estado autoritativo. Nem o corpus nem a teoria autorizam reduzir uma ambição à outra.

9.4 DIMENSÃO 3: PRONTIDÃO PERCEBIDA, AUTORIDADE E CONFIANÇA VISUAL

A terceira tese refere-se ao conceito de autoridade do painel. Nem toda atualização rápida produz confiança operacional. Um frontend pode se mover rapidamente e ainda assim permanecer dependente de projeções locais, sincronizações incompletas ou snapshots esparsos. É por isso que o relatório insiste em observar `snapshot_rate_per_second`, tempos de primeiro snapshot, tempos de estado significativo e métricas relacionadas ao fluxo de eventos visíveis.

Sob essa ótica, Pinot oferece um caso paradigmático. Sua superfície consultiva é excelente, mas a taxa autoritativa de snapshots observada no corpus é muito inferior à das demais stacks. Isso não elimina seu valor. Apenas impede que a velocidade de leitura seja interpretada como sinônimo de atualização autoritativa do painel. Druid, por contraste, mostra que um backend não precisa liderar todas as consultas para ainda assim sustentar um regime de visualização muito convincente quando a questão central é a retomada rápida do estado útil.

Essa distinção tem peso especial em ambientes contábeis e financeiros. O custo

de exibir rapidamente um estado não autoritativo pode ser maior do que a vantagem de exibi-lo cedo. A relevância do benchmark está justamente em não esconder essa ambiguidade.

9.5 DIMENSÃO 4: CUSTO OPERACIONAL E CUSTO COGNITIVO SÃO MÉTRICAS REAIS

Uma contribuição importante deste corpus é tratar custo operacional e custo cognitivo como parte do comportamento do backend, e não como ruído extra-experimental. Druid demanda disciplina de readiness, ingestão e coordenação de serviços. Pinot demanda maior governança da fronteira entre projeção local e reancoragem autoritativa. ClickHouse depende de uma boa separação entre derivação canônica e serving quente, além de desenho apropriado do read model. Materialize, por sua vez, cobra mais atenção à modelagem incremental, ao ciclo de bootstrap e à estabilidade da trilha inteira.

Esses custos não aparecem numa única consulta, mas aparecem no projeto real. Para uma equipe de engenharia, eles se traduzem em tempo de implementação, dificuldade de diagnóstico, sensibilidade a replays, qualidade de observabilidade e risco de regressão ao ajustar o workload. Logo, o artigo não deve tratar esses aspectos como notas de rodapé. Eles são parte da substância da comparação.

9.6 DIMENSÃO 5: CORRETUDE CONTÁBIL COMO FRONTEIRA MÍNIMA, NÃO COMO DIFERENCIAL SUFICIENTE

O fato de todas as stacks canônicas terem mantido `balance_sheet_difference` igual a zero é excelente para a qualidade epistemológica do relatório. Isso garante que a comparação principal não foi contaminada por uma solução que simplesmente "ganhou" por ter relaxado integridade semântica. Mas há um detalhe importante: corretude mínima não encerra o problema contábil. Preservar equilíbrio patrimonial é condição necessária; não é, por si só, prova suficiente de que a trilha completa de reconciliação, filtragem, atualização incremental e exibição final seja igualmente robusta em todas as dimensões.

Em outras palavras, a corretude observada delimita o campo do debate, mas não o resolve. Ela autoriza que se compare latência, convergência e operação sem suspeita imediata de fraude semântica. Ao mesmo tempo, impõe que a pesquisa futura continue observando outras formas de coerência: idempotência, comportamento sob correção tardia, consistência entre detalhamento e agregados, custo de replay e estabilidade sob filtros compostos mais agressivos.

9.7 DIMENSÃO 6: O PESO DO CORPUS AUXILIAR E DAS TENTATIVAS SUBSTITUÍDAS

Uma escolha importante desta versão ampliada é não reduzir o artigo apenas às rodadas finais canônicas. As tentativas substituídas e as rodadas auxiliares existem por uma razão: elas revelam fragilidades de preparação, aquecimento e maturação do experimento. A tentativa anterior de ClickHouse, com enorme concentração de falhas SQL, e as rodadas de validação de Materialize, com insucessos iniciais em consultas nativas, mostram que a comparação entre paradigmas não ocorre num vácuo. Ela depende de estabilização progressiva das trilhas e de eliminação incremental de erros de integração.

Esse dado tem duas implicações. A primeira é epistemológica: o artigo fica mais honesto quando registra a trilha de amadurecimento do corpus em vez de fingir que só existiram as quatro melhores rodadas. A segunda é prática: essas rodadas auxiliares são um mapa de onde vale a pena concentrar reruns de dupla conformação. O trabalho futuro não precisa repetir tudo cegamente; ele pode priorizar precisamente os pontos em que o corpus já mostrou fragilidade.

9.8 DIMENSÃO 7: RELAÇÃO ENTRE RECURSOS E DESEMPENHO

A leitura de recursos computacionais também relativiza conclusões rápidas. Druid e Pinot aparecem com consumo agregado de memória pico substancialmente superior ao de ClickHouse e Materialize, ao passo que ClickHouse lidera em CPU média agregada no corpus final. Isso impede leituras simplificadas do tipo "o melhor foi também o mais barato". A relação entre recursos e benefício depende do que se deseja otimizar. Se o objetivo for minimizar custo de memória local, certas stacks tornam-se menos atraentes. Se o objetivo for maximizar velocidade consultiva absoluta, um gasto maior de CPU pode ser perfeitamente aceitável.

O ponto principal é que o artigo deve explicitar a multidimensionalidade da escolha. Em projetos reais, não existe uma função objetivo única. Existem restrições orçamentárias, limites de infraestrutura, exigências de auditabilidade, tolerância a complexidade operacional e expectativas muito diferentes sobre o papel do frontend. É justamente por isso que um benchmark amplo como este precisa fornecer mais de uma forma de leitura.

9.9 UMA MATRIZ INTERPRETATIVA PARA DECISÃO ARQUITETURAL

À luz do corpus, é possível propor uma matriz interpretativa qualitativa. Para casos em que a prioridade máxima seja leitura direta rápida sobre estado quente com bom equilíbrio geral de integração, ClickHouse aparece como a escolha mais consistente do corpus final estabilizado. Para casos em que a camada consultiva pura, especialmente SQL, seja dominante e a equipe aceite maior protagonismo do frontend na continuidade visual, Pinot surge como candidato forte. Para contextos em que a organização aceite topologia multi-serviço mais exigente em troca de excelente prontidão percebida e boa cadência autoritativa, Druid se torna tecnicamente defensável. Para contextos em que o objetivo principal seja explorar proximidade semântica com incremental view maintenance e evoluir um caminho mais rigoroso entre stream e snapshot autoritativo, Materialize permanece como a linha de investigação mais fecunda, embora ainda não a mais madura no corpus final.

Essa matriz não substitui o benchmark; ela é o produto interpretativo do benchmark. Seu valor está em impedir decisões dogmáticas. Em vez de proclamar uma resposta universal, ela força a equipe a escolher explicitamente qual problema deseja resolver primeiro.

9.10 ENCERRAMENTO DA DISCUSSÃO COMPARATIVA

O ganho principal desta discussão ampliada é tornar claro que o relatório não está comparando apenas produtos, mas formas de organizar a distância entre evento e interface. Cada backend representa uma solução diferente para essa distância. Alguns a encurtam por velocidade de serving, outros por organização incremental do estado, outros por composição entre backend e frontend, outros por compromisso entre aquecimento, consulta e cadência autoritativa. Esse é o sentido mais forte da contribuição analítica do trabalho.

Quando o problema é formulado dessa forma, a diminuição do relatório seria um empobrecimento real: ela apagaria precisamente as nuances que tornam o benchmark útil. A versão ampliada precisa, portanto, preservar tabelas, preservar narrativa e acrescentar discussão. Esse é o princípio que orienta os capítulos finais e os apêndices técnicos que seguem.

10 IMPLICAÇÕES DE ENGENHARIA E PROGRAMA EXPERIMENTAL

10.1 O BENCHMARK COMO FERRAMENTA DE ARQUITETURA, NÃO APENAS DE MEDIÇÃO

Uma consequência importante deste trabalho é mostrar que o benchmark deixa de ser útil quando é tratado apenas como mecanismo de medição retrospectiva. O valor real do corpus cresce quando ele passa a orientar decisões de arquitetura futuras. Para isso, o artigo precisa ultrapassar a descrição dos números e entrar no terreno das implicações de engenharia: que tipo de equipe cada stack pressupõe, que classes de risco cada integração produz, que formas de observabilidade são indispensáveis, quais contratos de dados precisam permanecer estáveis e em que pontos o repositório ainda precisa amadurecer antes de repetir a bateria com poder analítico maior.

Essa leitura interessa porque sistemas near-real-time orientados por eventos costumam fracassar não apenas por escolherem o backend "errado", mas por não alinharem o backend escolhido à topologia do restante da stack. Um motor consultivo excelente pode degradar a experiência se a API, o gateway e o frontend forem incapazes de sustentar reconciliação robusta. Um mecanismo incremental elegante pode produzir atrito se o pipeline de bootstrap e observabilidade não estiver à altura da sua semântica temporal. Em ambos os casos, a decisão equivocada não está só no produto, mas na engenharia de integração.

10.2 CONTRATOS CANÔNICOS COMO PATRIMÔNIO DO EXPERIMENTO

O ativo mais valioso do repositório talvez não seja nenhum backend específico, mas o contrato canônico que o benchmark conseguiu estabilizar. O stream `ledger-entries-v1`, a disciplina de dupla entrada e a separação entre evento bruto e entry contábil derivada constituem a espinha dorsal da comparabilidade. Sem esse contrato, cada backend precisaria ser comparado sobre pipelines semânticos distintos, e o benchmark degeneraria em competição entre interpretações diferentes do domínio.

Para a engenharia futura, isso significa que qualquer evolução do repositório deve proteger agressivamente essa camada. É admissível mudar APIs, gateways, modelos auxiliares, caches, filtros, views materializadas e frontends especializados. Não

é admissível fragilizar a camada canônica que torna as comparações defensáveis. Em termos práticos, isso implica versionamento rigoroso de schemas, testes de compatibilidade de eventos, validação contínua da contabilidade derivada e capacidade de replay auditável.

10.3 OBSERVABILIDADE COMO REQUISITO DE COMPARABILIDADE

Outra implicação forte do corpus é a necessidade de elevar observabilidade ao estatuto de requisito metodológico. Não basta que uma stack "funcione"; ela precisa funcionar de modo auditável. O problema das coberturas parciais de debug em algumas rodadas deixa isso muito claro. Quando faltam snapshots finais consistentes, o artigo perde poder explicativo. Não se perde apenas um detalhe operacional; perde-se capacidade de reconstruir, a posteriori, por que o sistema convergiu, divergiu, aqueceu lentamente ou falhou numa consulta específica.

O programa experimental futuro deveria, portanto, estabelecer um princípio simples: toda superfície usada na argumentação do artigo deve ter superfície correspondente de observabilidade. Se uma API entra na comparação, deve haver payload final capturável. Se uma estratégia de convergência depende de gateway, deve haver trilha de estado suficiente para reconstruir como esse gateway se comportou. Se um backend depende fortemente de bootstrap ou de ingestão, os estágios desse bootstrap precisam virar métricas de primeira classe.

10.4 REPLAYS, CORREÇÕES TARDIAS E SEMÂNTICA TEMPORAL

O corpus atual é sólido para a primeira bateria conclusiva, mas ainda insuficiente para esgotar a dimensão temporal do problema. Uma agenda de engenharia realmente ambiciosa deve introduzir cenários controlados de replay, correção tardia, retrações, reprocessamento parcial e divergência entre dados de catálogo e histórico de ledger. Essas situações não são patológicas no domínio; são parte normal da vida de sistemas financeiros reais.

O que o benchmark fez até aqui foi provar que vale a pena tratar essas questões como objeto central. O que falta fazer é incorporá-las ao protocolo, com cenários específicos e métricas dedicadas. Para ClickHouse, isso permitiria observar quanto da robustez depende de estado já estabilizado. Para Druid, revelaria o custo de rehidratação e recompostura da ingestão em cenários mais próximos da operação real. Para Pinot, testaria a robustez da combinação entre projeção local e reancoragem periódica.

Para Materialize, ofereceria o palco ideal para observar em que medida a vantagem conceitual incremental se converte em vantagem empírica quando o domínio efetivamente exige atualização semântica difícil.

10.5 ENGENHARIA DE FRONTEND E O ERRO DE SUBESTIMAR A BORDA

Uma das lições mais importantes do corpus é que o frontend não pode ser tratado como apêndice cosmético do benchmark. Em várias stacks, parte crítica da experiência percebida depende de como a borda consome websocket, projeta eventos, decide quando trocar o estado exibido e quando aceitar um snapshot autoritativo como suficiente. Ignorar essa camada seria mutilar o fenômeno medido.

Do ponto de vista de produto, isso implica que o benchmark futuro deveria versionar também políticas de frontend: critérios de estado significativo, modos de ressincronização, políticas de deduplicação, tolerância a vazio transitório, fallback para polling e tratamento de desconexão. Sem isso, corre-se o risco de atribuir ao banco diferenças que, na prática, pertencem à política de UI e de gateway.

10.6 UMA AGENDA OBJETIVA PARA RERUNS DE DUPLA CONFORMAÇÃO

O pedido de dupla conformação faz sentido e o corpus já indica onde essa energia deve ser gasta primeiro. A prioridade técnica imediata é repetir a bateria final com foco especial em Druid e ClickHouse. Druid precisa validar novamente a consulta `filtered_entries` e confirmar se os 503 iniciais e as falhas parciais de API pertencem de fato ao aquecimento esperado ou a uma fragilidade ainda não totalmente resolvida. ClickHouse, por sua vez, precisa repetir a trajetória que levou da tentativa substituída com falhas SQL massivas à rodada canônica final limpa, para demonstrar que a melhoria foi estabilização reprodutível e não apenas uma boa execução isolada.

Materialize também merece reruns, mas por razão distinta. No seu caso, o objetivo não é apenas verificar uma falha pontual, e sim medir se a trilha incremental consegue ganhar maturidade prática sem perder a contribuição conceitual. Para Pinot, o foco principal dos reruns não deveria ser latência consultiva, já suficientemente forte no corpus, mas sim a cadência autoritativa de snapshots e a qualidade da auditabilidade final.

10.7 EXPANSÃO DO PROTOCOLO: ALÉM DE UMA ÚNICA RODADA CANÔNICA

Para que o relatório cresça em profundidade científica, o protocolo precisa amadurecer de uma "bateria final bem-sucedida" para um desenho com repetições deliberadas, cenários diferenciados e análise de variância. Isso não significa transformar o trabalho num tratado estatístico desconectado da engenharia. Significa apenas reconhecer que a força atual do corpus está na rastreabilidade, e que o próximo salto metodológico natural é combinar essa rastreabilidade com repetição suficiente para separar sinal de variabilidade local.

Um desenho plausível incluiria pelo menos três famílias de cenário. A primeira manteria o caso atual de serving near-real-time contínuo. A segunda introduziria replay controlado e correções tardias. A terceira variaria a composição de filtros, cardinalidades e tamanho de payload do workspace. Com isso, o artigo deixaria de registrar apenas "como as stacks se comportaram num caso" e passaria a mostrar "como os paradigmas se movem quando a pressão do domínio muda".

10.8 O PAPEL DO RELATÓRIO AMPLIADO DENTRO DO REPOSITÓRIO

À medida que o relatório cresce, ele deixa de ser apenas documentação de resultados e passa a funcionar como camada de memória técnica do projeto. Isso é particularmente importante em repositórios experimentais longos, nos quais decisões metodológicas se perdem facilmente com o tempo. Um artigo técnico extenso, desde que ancorado em artefatos reais, cumpre também a função de consolidar racionalidade histórica: por que o protocolo foi desenhado assim, o que se tentou antes, quais trilhas falharam, por que certas correções foram introduzidas, e quais hipóteses ainda permanecem abertas.

Nesse sentido, aumentar a extensão do documento não é luxo estilístico. É uma forma de preservar contexto técnico que, de outro modo, ficaria disperso entre scripts, commits, observações de execução e memória tácita do desenvolvedor.

10.9 FECHO DO PROGRAMA EXPERIMENTAL

O programa que emerge deste capítulo é claro. Primeiro, preservar a comparabilidade semântica do contrato canônico. Segundo, elevar observabilidade a requisito de primeira ordem. Terceiro, repetir o corpus final onde houve falha, substituição ou fragilidade de auditabilidade. Quarto, introduzir cenários temporais e operacionais mais duros que aproximem o benchmark da realidade de sistemas financeiros vivos. Quinto,

tratar frontend, gateway e backend como partes indissociáveis de uma arquitetura de serving.

Se o repositório seguir esse programa, o relatório deixará de ser apenas uma fotografia competente de uma bateria final e se tornará uma referência técnica ampla sobre como comparar paradigmas de serving para dashboards gerenciais orientados por eventos. Esse é o horizonte que justifica a expansão do texto e a reestruturação do documento nesta versão.

11 MATRIZ DE DECISÃO ARQUITETURAL

11.1 POR QUE ESTE CAPÍTULO EXISTE

Um problema recorrente em relatórios de benchmark é o salto brusco entre evidência e prescrição. Primeiro apresenta-se uma massa de números; depois, quase sem mediação, declara-se um "vencedor" para uso prático. Essa passagem raramente é inocente. Ela costuma embutir prioridades implícitas que não foram declaradas: preferência por throughput, aversão a complexidade operacional, valorização de latência percebida, sensibilidade a custo de memória, exigência de auditabilidade ou tolerância a protagonismo do frontend. Como este relatório explicitamente recusou a ideia de vencedor universal, torna-se necessário oferecer uma camada final de decisão arquitetural capaz de ligar métricas, paradigmas e contextos de uso de forma mais transparente.

Este capítulo cumpre essa função. Ele não substitui a discussão técnica anterior e tampouco reduz a decisão a uma tabela de notas. Seu objetivo é mostrar como os resultados podem ser traduzidos em critérios de escolha sob diferentes prioridades organizacionais. Em outras palavras, o capítulo converte o corpus em ferramenta de decisão, sem trair a multidimensionalidade que o próprio benchmark revelou.

11.2 EIXO DECISÓRIO 1: QUANDO A PRIORIDADE DOMINANTE É CONSULTA RÁPIDA DE PAINEL

Em organizações cujo problema principal é servir rapidamente resumos executivos, painéis consolidados e consultas interativas frequentes sobre um estado já aquecido, o eixo dominante tende a ser a velocidade consultiva. Nesse cenário, a pergunta central não é tanto como reduzir a distância conceitual entre stream e snapshot, mas como garantir que o painel responda com baixa latência sob operação normal. O corpus aponta com força para duas candidaturas principais: ClickHouse e Pinot.

ClickHouse aparece como solução de maior equilíbrio geral nesse eixo. Sua latência HTTP foi muito forte, sua superfície SQL permaneceu extremamente competitiva e a cadência autoritativa de snapshots não entrou em colapso para sustentar essa velocidade. Isso o torna especialmente atraente para equipes que desejam alto desempenho de leitura sem aceitar degradação severa de reconvergência visual. Além disso, a topologia da trilha no repositório é relativamente inteligível: writer canônico, serving analítico, gateway e frontend conseguem cooperar sem exigir que o frontend assuma a

maior parte da inteligência temporal.

Pinot, por outro lado, surge como solução particularmente poderosa quando a prioridade é maximizar a agressividade do serving SQL e HTTP puro. O backend demonstrou excelente comportamento em consultas sintéticas e em partes importantes da API. Contudo, a decisão por Pinot só é integralmente defensável quando a organização assume conscientemente seu preço arquitetural: mais protagonismo do frontend, maior importância da política de reancoragem e menor cadência autoritativa observada no corpus final. Se esse preço for aceitável, Pinot torna-se opção muito forte. Se não for, sua superioridade consultiva talvez não compense a distância para o estado global autoritativo do painel.

11.3 EIXO DECISÓRIO 2: QUANDO A PRIORIDADE DOMINANTE É PRONTIDÃO PERCEBIDA DO FRONTEND

Há cenários em que o principal problema de negócio não é o menor p95 absoluto, mas o tempo necessário para que o operador volte a enxergar um painel útil logo após o arranque, a reconexão ou o início da sessão. Em tais contextos, métricas como `frontend_time_to_first_` e `snapshot_rate_per_second` ganham peso extraordinário. O corpus mostra que, nesse eixo, Druid precisa ser levado muito a sério.

Druid oferece talvez o melhor exemplo de como experiência percebida e custo consultivo podem divergir. Sua API não lidera a comparação, e certas superfícies SQL permaneceram frágeis. Mesmo assim, a retomada visual e a cadência autoritativa do painel aparecem como pontos fortes relevantes. Isso sugere que Druid é tecnicamente atraente quando a organização precisa de sensação operacional de tempo real muito cedo e aceita pagar com topologia mais exigente e aquecimento mais delicado.

ClickHouse continua sendo forte também nesse eixo, embora menos brilhante que Druid no instante inicial percebido. A diferença é que ClickHouse entrega esse resultado em uma configuração mais equilibrada entre consulta, snapshots e estabilidade final. A decisão entre os dois depende, portanto, do quanto a equipe valoriza o arranque perceptual puro em comparação com previsibilidade operacional e completude de superfície.

11.4 EIXO DECISÓRIO 3: QUANDO A PRIORIDADE DOMINANTE É GOVERNANÇA DO ESTADO AUTORITATIVO

Em certas organizações, a maior preocupação não é apenas "parecer rápido", mas assegurar que o painel reflita, com a menor distância possível, um estado autoritativo consistente e semanticamente controlado. Esse eixo é especialmente importante em domínios regulados, contábeis, financeiros ou operacionais críticos. Nesses casos, a arquitetura que governa a autoridade do snapshot torna-se tão importante quanto a velocidade consultiva.

Sob essa ótica, Materialize reaparece com força analítica, mesmo sem liderar consultas. O backend não foi o mais rápido, mas continua sendo o caso do corpus que melhor encarna a ambição de aproximar a manutenção do estado derivado do próprio coração do sistema. Isso não é pequeno. Significa que, para organizações cuja estratégia de longo prazo exige tratar incrementalidade e temporalidade como virtudes centrais do backend, Materialize pode representar a trilha de maior valor evolutivo, ainda que não seja a melhor resposta imediata para o workload já testado.

ClickHouse e Druid também podem sustentar autoridade do painel, mas o fazem por composição. Em ambos, parte importante da solução continua distribuída entre API, gateway e frontend. Isso é totalmente aceitável em muitas arquiteturas. Apenas muda a natureza do controle. Em vez de concentrar a autoridade incremental no backend, a organização passa a governá-la através de contratos entre múltiplas camadas.

11.5 EIXO DECISÓRIO 4: QUANDO A PRIORIDADE DOMINANTE É SIMPLICIDADE OPERACIONAL LOCAL

Há equipes cujo principal gargalo não está no hardware nem no modelo conceitual, mas na capacidade prática de operar, depurar e repetir a stack. Nesses ambientes, custo operacional local passa a ser variável central. O benchmark mostra com clareza que esse custo não é homogêneo.

Druid é a stack mais explicitamente onerosa nesse quesito, porque sua topologia distribuída torna o aquecimento e a coordenação muito mais sensíveis. Pinot também demanda cuidados relevantes, embora em outro formato, sobretudo na relação entre backend e política de frontend. Materialize cobra preço de natureza distinta: não tanto pela multiplicidade de serviços, mas pela sofisticação do paradigma incremental e pela instabilidade ainda observável na trilha atual. ClickHouse surge, no corpus final estabilizado, como a solução que melhor combina alta performance com uma sensação

relativamente mais controlável de operação, desde que o pipeline canônico anterior a ele esteja bem resolvido.

Em linguagem decisória, isso significa que ClickHouse tende a ser a alternativa de menor atrito para equipes que desejam resultado forte sem transformar o benchmark inteiro em projeto de operação distribuída. Pinot pode ser igualmente atraente em times maduros em frontend e serving interativo. Druid e Materialize, por sua vez, exigem disposição explícita para investir em maturação operacional e observabilidade.

11.6 EIXO DECISÓRIO 5: QUANDO A PRIORIDADE DOMINANTE É APRENDIZAGEM ESTRATÉGICA DA ORGANIZAÇÃO

Nem toda escolha de backend é decisão de curto prazo. Algumas organizações usam projetos experimentais para aprender sobre paradigmas, preparar capacidades internas e decidir em que direção tecnológica desejam evoluir. Nesse tipo de cenário, o backend "mais rápido hoje" nem sempre é o backend "mais valioso para aprender".

Sob esse critério, Materialize ocupa posição privilegiada. Ele força a equipe a pensar em incremental view maintenance, fronteiras temporais, autoridade do snapshot e custo semântico da modelagem relacional. Mesmo que a trilha atual ainda esteja aquém de uma solução prática madura para o corpus medido, o backend amplia o repertório conceitual do projeto. Druid também tem valor estratégico semelhante, embora por outra razão: ele expõe, com muita clareza, o custo e o poder de uma topologia de serving analítico distribuído orientada a eventos.

ClickHouse e Pinot, por outro lado, oferecem aprendizado estratégico mais ligado a excelência de serving, modelagem quente e eficiência consultiva. A escolha entre aprender mais sobre incrementalidade ou mais sobre leitura quente não é técnica apenas; é também organizacional e de portfólio.

11.7 CENÁRIOS DE DECISÃO SINTETIZADOS

Se uma organização precisa escolher um backend para um painel gerencial com forte exigência de leitura rápida, baixa tolerância a complexidade distribuída adicional e interesse em boa cadência de snapshots autoritativos, o corpus favorece ClickHouse como primeira escolha concreta. Se a organização deseja maximizar performance consultiva e já domina políticas sofisticadas de frontend e gateway, Pinot passa a ser opção muito atraente. Se o objetivo é privilegiar experiência percebida e recuperação visual rápida, aceitando maior disciplina de operação, Druid ganha força. Se o horizonte é construir

uma linha de evolução incremental mais profunda, mesmo com maior custo atual, Materialize continua sendo a investigação mais promissora.

É crucial observar que essas recomendações não se excluem mutuamente. Um laboratório de arquitetura maduro pode perfeitamente adotar ClickHouse para produção imediata, manter Pinot como referência de performance consultiva, usar Druid como linha de comparação para topologias de ingestão e sustaining analítico, e preservar Materialize como trilha estratégica de pesquisa e evolução. O relatório só faz sentido pleno quando lido como catálogo de possibilidades tecnicamente informadas, não como cerimônia de coroação de um único produto.

11.8 O QUE A MATRIZ DE DECISÃO CORRIGE EM RELAÇÃO A BENCHMARKS SIMPLIFICADOS

Esta matriz corrige dois vícios comuns. O primeiro é o vício do monocritério, em que a decisão é tomada como se todas as organizações tivessem a mesma função objetivo. O segundo é o vício da abstração excessiva, em que a arquitetura real da stack é apagada e os produtos passam a ser comparados como se existissem fora das APIs, gateways, frontends e contratos que lhes dão sentido no experimento.

Ao explicitar critérios, o capítulo torna mais difícil o uso indevido do benchmark. Ninguém poderá citar o relatório honestamente dizendo apenas "o backend X venceu", sem também dizer em qual eixo ele venceu, qual custo pagou e quais condições de integração tornaram esse resultado possível.

11.9 CENÁRIOS EXEMPLARES DE DECISÃO

Para tornar a matriz ainda mais concreta, vale imaginar alguns cenários exemplares. O primeiro é o de uma empresa que já possui pipeline contábil canônico estável, equipe pequena de backend e forte necessidade de dashboards executivos responsivos com baixa tolerância a crescimento de complexidade operacional. Nesse caso, a combinação de alto desempenho consultivo, cadência autoritativa robusta e integração relativamente mais controlável torna ClickHouse a aposta mais prudente no corpus atual. A decisão não decorre apenas da velocidade, mas do fato de que o backend oferece ótima resposta sem exigir, no estado presente do repositório, o grau de reengenharia que outras trilhas ainda pedem.

O segundo cenário é o de uma organização digital cuja cultura de produto aceita um frontend muito ativo, com boa engenharia de estado local, forte disciplina de reancoragem e prioridade máxima em experiência de navegação e exploração interativa. Nessa configuração, Pinot ganha valor adicional. Sua superioridade consultiva pode

ser explorada mais plenamente quando a equipe não vê a inteligência do cliente como problema, mas como parte deliberada da solução arquitetural.

O terceiro cenário é o de uma operação que precisa de retomada perceptual extremamente rápida após reconexões, reinícios ou oscilações de readiness, e que já tem maturidade razoável para operar topologias mais distribuídas. Aqui, Druid pode ser estrategicamente vantajoso. O backend passa a ser justificado menos por vencer tabelas de consulta e mais por oferecer um tipo de compromisso entre fluxo, snapshots e percepção de utilidade que outras stacks não entregam da mesma forma.

O quarto cenário é o de uma organização que deseja usar o projeto não apenas para servir dashboards atuais, mas para formar competência interna em manutenção incremental, temporalidade e semântica de estado derivado mais próxima do stream. Para esse contexto, Materialize permanece extremamente valioso, mesmo sem liderança prática nas métricas finais do corpus. O ganho não é imediato em p95; é estratégico na capacidade de experimentar um paradigma que pode tornar-se cada vez mais relevante à medida que o domínio exigir replay, correção tardia e views incrementais mais ricas.

11.10 TRADE-OFFS QUE NÃO DEVEM SER OMITIDOS NO USO DO RELATÓRIO

Uma matriz de decisão só é honesta quando explicita também o que ela não resolve. Este relatório, por mais ampliado que esteja, ainda se apoia em um único host local, em budgets de fase controlados e em um conjunto finito de cenários. Isso significa que as recomendações aqui derivadas devem ser interpretadas como decisões informadas pelo corpus, não como leis gerais sobre os produtos avaliados. Há ainda espaço real para mudança de posição relativa quando o workload, a infraestrutura ou a política de frontend se alterarem.

Também não seria correto usar a matriz como desculpa para apagar o papel do restante da stack. Nenhum backend deste relatório existe sozinho. Todos dependem de writer, API, gateway, frontend, health checks e observabilidade. Se uma equipe tentar "importar" a conclusão do relatório sem importar a disciplina arquitetural correspondente, o documento será usado de forma incorreta. A matriz de decisão é uma ferramenta para escolher soluções dentro de um sistema, não um atalho para pular o desenho do sistema.

Há, por fim, um último trade-off que precisa ser verbalizado. Quanto mais se busca uma resposta universal, menos útil o benchmark se torna. O valor real do relatório está justamente em obrigar a organização a dizer o que deseja otimizar. Essa obrigação pode ser desconfortável, mas é tecnicamente saudável. Ela evita que a decisão sobre backend seja tomada no terreno vago do gosto pessoal ou da reputação de mercado e

a desloca para o terreno explícito dos critérios, dos riscos e das prioridades.

11.11 FECHO DA MATRIZ DECISÓRIA

A utilidade final da matriz não está em encerrar o debate, mas em discipliná-lo. Ela obriga cada decisão a declarar seu critério dominante, seu apetite por complexidade, sua relação com a autoridade do estado e sua prioridade entre consulta rápida, prontidão percebida e aprendizagem estratégica. Esse é exatamente o tipo de ganho que um documento técnico extenso deve oferecer. Sem isso, a ampliação do relatório seria só volume; com isso, ela se torna estrutura de decisão.

12 CONCLUSÃO

12.1 RESPOSTA À PERGUNTA DO RELATÓRIO

O experimento realizado permite responder, com base empírica real e rastreável, que a escolha de backend para dashboards gerenciais orientados por eventos não admite resposta universal. O backend mais adequado depende da dimensão que se deseja otimizar e do custo aceitável nas demais dimensões.

Se o critério dominante for latência de API sob consultas de painel, ClickHouse apresentou o melhor comportamento observado neste corpus. Se o foco principal recair sobre latência SQL nativa, Pinot obteve os menores p95 medidos. Se a prioridade for combinar prontidão visual muito baixa com cadência consistente de snapshots autoritativos, Druid emergiu como o melhor compromisso observado entre convergência e custo de leitura. Materialize, por sua vez, demonstrou a capacidade de dissociar a utilidade inicial do frontend do custo total de consulta, mas o fez com a maior latência interna e com a maior volatilidade operacional dentre as stacks comparadas.

12.2 CONTRIBUIÇÃO ANALÍTICA PRINCIPAL

O resultado mais relevante do trabalho não é a proclamação de um vencedor absoluto, mas a demonstração de que a avaliação de backends para serving near-real-time precisa ser multidimensional. A combinação de API, SQL, websocket, health e corretude contábil mostrou que conclusões baseadas em uma única família de métricas podem ser profundamente enganosas.

Esse ponto aparece com clareza em pelo menos dois casos. Pinot foi excelente no plano consultivo, mas exibiu cadence de snapshots muito inferior à das demais stacks. Materialize, em sentido oposto, sustentou convergência visual competitiva mesmo sob latências SQL e HTTP muito mais elevadas. Tais contrastes seriam invisíveis em um benchmark restrito a p95 de consulta.

12.3 IMPLICAÇÕES ARQUITETURAIS

As evidências coletadas favorecem uma leitura arquitetural específica. Motores analíticos voltados a serving direto tendem a maximizar performance de leitura imediata. Sistemas com projeções incrementais podem reduzir o intervalo entre evento e utilidade percebida, mas não necessariamente minimizam o custo das consultas subjacentes. Sistemas intermediários, como Druid no corpus observado, podem oferecer equilíbrio

pragmático entre as duas frentes, desde que a orquestração operacional respeite suas exigências de readiness e ingestão.

Essa conclusão é especialmente relevante para sistemas contábeis e gerenciais, nos quais a confiabilidade semântica do estado exibido importa tanto quanto a velocidade. A manutenção de `balance_sheet_difference` igual a 0.0 em todas as stacks observadas mostra que o benchmark preservou a dimensão substantiva do problema, e não apenas sua superfície de performance.

12.4 LIMITES E AGENDA IMEDIATA

As conclusões deste relatório são fortes como descrição do corpus coletado, mas deliberadamente moderadas como generalização. O estudo ainda carece de repetições adicionais por backend, variação controlada de carga, consolidação estatística sobre múltiplas rodadas e exploração mais sistemática das métricas de recursos computacionais. Também permanece aberta a investigação de ajustes de configuração que possam alterar substancialmente a posição relativa de cada stack.

O passo natural seguinte não é reescrever a taxonomia dos resultados, mas expandir a amostra mantendo o mesmo protocolo auditável. Isso inclui repetir a bateria conclusiva, tratar variância inter-rodada, incorporar análise transversal de CPU e memória e investigar por que determinados backends dissociam tão fortemente latência consultiva e convergência visual.

12.5 FECHO FINAL

O relatório conclui, portanto, que o problema de serving para dashboards gerenciais em tempo real deve ser formulado como problema de sistema completo. Evento, trilha contábil, backend, API, gateway e frontend compõem uma cadeia única de valor observável. A stack tecnicamente preferível será aquela que melhor alinhar, para um contexto específico, desempenho consultivo, estabilidade operacional, cadência de atualização e preservação da corretude contábil.

No corpus empírico aqui obtido, ClickHouse foi a melhor opção geral para latência de leitura, Pinot foi a melhor opção para SQL puro, Druid apresentou o compromisso mais equilibrado entre rapidez percebida e cadência autoritativa, e Materialize permaneceu como a stack mais interessante para explorar semântica incremental, embora ainda não como a mais eficiente para o workload medido. Essa é a conclusão substantiva que os dados reais autorizam neste estado do projeto.

A APENDICE A. PROTOCOLO DE COLETA

A.1 PROPOSITO

Este apendice resume o protocolo de coleta adotado para o benchmark individualizado dos backends do projeto, preservando sua funcao como referencia operacional e metodologica do manuscrito.

A.2 PRINCIPIOS

1. Cada rodada ativa exatamente um backend em `ACTIVE_STACKS`.
2. Nenhuma comparacao quantitativa deve misturar medicoes feitas com stacks concorrentes.
3. Cada rodada deve durar no maximo 10 minutos.
4. Resultados devem sempre registrar commit Git, backend, cenario, duracao e configuracao efetiva do ambiente.

A.3 FLUXO PADRAO

1. Preparacao do ambiente e limpeza de residuos.
2. Subida da stack e populacao do cenario.
3. Espera por health minimo e aquecimento.
4. Coleta de API, SQL, websocket, health e recursos.
5. Consolidacao dos artefatos e teardown.

A.4 SCRIPTS IMPLEMENTADOS

- `scripts/benchmark/run_round.sh`
- `scripts/benchmark/run_clickhouse_round.sh`
- `scripts/benchmark/run_druid_round.sh`
- `scripts/benchmark/run_pinot_round.sh`
- `scripts/benchmark/run_materialize_round.sh`
- `scripts/benchmark/collect_api_latencies.py`
- `scripts/benchmark/collect_sql_latencies.py`
- `scripts/benchmark/collect_ws_convergence.mjs`

- `scripts/benchmark/collect_container_stats.py`
- `scripts/benchmark/collect_health_timeline.py`
- `scripts/benchmark/collect_debug_snapshots.py`
- `scripts/benchmark/consolidate_round_results.py`

A.5 OBSERVACAO METODOLOGICA

O protocolo nao busca apenas medir consultas. Ele mede o sistema completo, incluindo bootstrap, saude operacional, reconvergencia visual e corretude contabil.

B APENDICE B. ESPECIFICACAO CANONICA DOS ARTEFATOS

Cada rodada de benchmark deve produzir um conjunto uniforme de arquivos, permitindo auditoria, reprocessamento e comparacao longitudinal.

B.1 ARTEFATOS OBRIGATORIOS

- `metadata.json`
- `api_latencies.json`
- `sql_latencies.json`
- `ws_metrics.json`
- `resources.json`
- `health_timeline.json`
- `debug_snapshots.json`
- `round_report.json`
- `summary.csv`

B.2 ESTRUTURA LOGICA DO CONSOLIDADO

O artefato canonico de rodada e `round_report.json`, cuja estrutura minima e:

```
{
  "metadata": {},
  "api_latencies": {},
  "sql_latencies": {},
  "ws_metrics": {},
  "resources": {},
  "health_timeline": {},
  "debug_snapshots": {},
  "derived_metrics": {}
}
```

B.3 COLUNAS TRANSVERSAIS DO CSV

O `summary.csv` deve conter ao menos as seguintes colunas:

- `round_id`
- `commit`
- `backend`
- `scenario`
- `run_number`
- `started_at`
- `duration_seconds`
- `api_summary_p95_ms`
- `api_workspace_p95_ms`
- `sql_entries_count_p95_ms`
- `sql_summary_by_role_p95_ms`
- `frontend_time_to_first_meaningful_state_ms`
- `snapshot_rate_per_second`
- `entry_rate_per_second`
- `balance_sheet_difference`
- `health_transition_total`

B.4 REGRA SEMANTICA

Todos os campos devem refletir valores realmente observados. Quando um valor nao estiver disponivel, ele deve aparecer como `null` no JSON e permanecer vazio no CSV. Nao se admitem placeholders textuais em artefatos experimentais finais.

C APÊNDICE C - DOSSIÊS OPERACIONAIS POR BACKEND

C.1 FINALIDADE DOS DOSSIÊS

Este apêndice reúne uma leitura operacional pormenorizada de cada trilha do benchmark. Seu objetivo não é repetir as tabelas do corpo principal, mas registrar, em linguagem de engenharia, os elementos que tipicamente ficam implícitos quando um relatório é condensado demais: pressupostos de integração, pontos de fragilidade, consequências das escolhas de modelagem e indícios de maturidade ou imaturidade operacional encontrados ao longo da coleta. Em um documento mais curto, tais observações tenderiam a desaparecer. Aqui, elas são preservadas porque ajudam a transformar o corpus em memória técnica reutilizável.

C.2 DOSSIÊ CLICKHOUSE

C.2.1 Papel da stack dentro do repositório

ClickHouse ocupa, no repositório, a posição de motor colunar de leitura quente alimentado por uma trilha canônica já semanticamente resolvida a montante. O produtor gera eventos de negócio; o storage writer realiza a derivação contábil; o backend recebe entries prontas para serving; a API especializada reorganiza o acesso em contratos adequados ao dashboard; o gateway harmoniza a atualização com a experiência visual. Esse arranjo permite que ClickHouse seja medido em um regime próximo daquilo em que costuma ser mais forte: baixa latência de leitura sobre estruturas tabulares quentes, com forte apoio da aplicação periférica para gerir reconciliação e continuidade de interface.

C.2.2 O que a stack parece fazer muito bem

Os artefatos sugerem que ClickHouse foi particularmente competente em resumos e leituras recorrentes de painel. O bom desempenho do endpoint `summary` e a posição competitiva em `workspace` e consultas SQL reforçam a hipótese de que o read model foi bem alinhado ao padrão de agregações do domínio. Esse alinhamento é importante porque afasta a interpretação simplista de que "o banco é rápido por natureza". O banco foi rápido porque havia combinação entre sua estrutura interna e a forma como a camada de aplicação o explorou.

Outra virtude relevante é a convivência relativamente saudável entre consulta rápida e cadência autoritativa de snapshots. Em muitos cenários, motores hot-analytic correm o risco de ficar excelentes em consulta e medianos em reconvergência visual. A trilha ClickHouse mostrou que isso não é inevitável quando o gateway está bem desenhado e quando o frontend não depende de um único mecanismo para parecer vivo.

C.2.3 O que a stack exige da engenharia

A exigência principal da trilha ClickHouse está em outro lugar: ela pressupõe uma boa separação de responsabilidades. Se o writer canônico degradar, se a API errar o desenho do read model, se o gateway perder disciplina de ressincronização ou se o frontend for permissivo demais com estados transitórios, a boa performance do banco deixa de ser suficiente. Em outras palavras, ClickHouse recompensa engenharia periférica disciplinada. Não é uma solução que magicamente internaliza a semântica do sistema inteiro.

Também é importante notar que a tentativa substituída com falhas SQL massivas é parte do dossiê. Ela mostra que a stack pode degradar violentamente quando alguma camada do preparo experimental falha. Portanto, a imagem correta de ClickHouse no corpus não é a de backend infalível, e sim a de backend muito competitivo quando a trilha ao redor já foi devidamente estabilizada.

C.2.4 Interpretação prática

Para equipes que já dispõem de pipeline canônico confiável, possuem boa disciplina de modelagem tabular e valorizam forte desempenho de leitura com custo operacional relativamente previsível, ClickHouse emerge como escolha extremamente séria. A principal cautela é não confundir velocidade consultiva com suficiência arquitetural. O backend não elimina o trabalho de reconciliação; ele funciona muito bem quando esse trabalho já foi corretamente redistribuído pelas demais camadas do sistema.

C.3 DOSSIÊ DRUID

C.3.1 Papel da stack dentro do repositório

Druid aparece, no repositório, como a trilha em que a topologia distribuída pesa mais explicitamente sobre o comportamento observado. Broker, router, middlemanager, coordinator, overlord, metadata store e serviços auxiliares constituem uma cadeia de prontidão mais longa e mais sensível do que a das demais stacks. Isso torna Druid particularmente ilustrativo para um ponto metodológico essencial: em certos backends, a unidade real do benchmark é a topologia distribuída inteira.

C.3.2 O que a stack parece fazer muito bem

Apesar do custo de aquecimento e dos episódios iniciais de indisponibilidade parcial, Druid mostrou desempenho notável na dimensão de experiência percebida. O frontend atinge estado útil cedo e mantém cadência de snapshots compatível com ClickHouse e Materialize. Essa propriedade é relevante porque prova que o backend pode continuar interessante mesmo quando não lidera as superfícies consultivas puras. Em cenários em que o dashboard precisa "voltar a respirar" rapidamente após o arranque, essa característica pode ser mais importante do que a diferença entre dezenas ou centenas de milissegundos em uma query específica.

C.3.3 Fragilidades operacionais registradas

O corpus também deixa claras as fragilidades. Os 503 iniciais de API, a transição de warming up, a dependência entre ingestão e serving e a falha da consulta `filtered_entries` demonstram que a trilha ainda exige cuidados importantes para garantir comparabilidade total entre todas as superfícies. Em ambientes reais, esse tipo de sensibilidade não desaparece por ser chamado de "questão operacional". Ele consome tempo de engenharia, amplia o custo de rerun e reduz a previsibilidade do resultado.

C.3.4 Interpretação prática

Druid parece adequado para equipes que aceitam topologia mais exigente em troca de uma combinação interessante entre análise quente, ingestão orientada a eventos e boa experiência inicial do painel. O backend não aparece como solução minimalista nem como resposta universal, mas como alternativa tecnicamente respeitável quando se deseja boa atualização percebida e quando se tolera uma infraestrutura mais complexa para chegar a esse resultado.

C.4 DOSSIÊ PINOT

C.4.1 Papel da stack dentro do repositório

Pinot ocupa o lugar da stack mais consultivamente agressiva do corpus. Sua integração no repositório favorece um modo de operação em que o backend serve consultas muito rápidas e o frontend participa ativamente da continuidade visual. Nessa arquitetura, o banco não precisa ser o único mecanismo de sensação de tempo real; ele atua como âncora rápida de leitura autoritativa, enquanto o cliente administra parte da fluidez local.

C.4.2 O que a stack parece fazer muito bem

O corpus é inequívoco quanto ao seu maior mérito: Pinot venceu as principais consultas SQL e foi excelente em parte importante da superfície HTTP. Isso o torna particularmente atraente para workloads de serving interativo, filtros frequentes e respostas rápidas em painéis operacionais com forte orientação a exploração. O backend também parece confortável em consultas sintéticas recorrentes de baixa cardinalidade, o que reforça sua vocação de OLAP operacional responsivo.

C.4.3 O ponto de tensão da trilha

O problema central não está na consulta, mas na distância entre consulta e reconciliação autoritativa. A taxa de snapshots por segundo observada no corpus final é muito inferior à das demais stacks. Isso cria uma assimetria que o relatório não pode ignorar. O backend parece ótimo para responder; a questão em aberto é o quanto essa excelência se transforma em painel plenamente autoritativo com a mesma frequência. Some-se a isso a cobertura limitada dos snapshots finais de debug, e aparece um tipo de custo menos óbvio: custo de auditabilidade.

C.4.4 Interpretação prática

Pinot é forte quando a organização aceita colocar mais inteligência na borda e deseja maximizar performance de serving direto. Para equipes que tratam o frontend como ator semântico de primeira linha e possuem boa disciplina de reconciliação, isso pode ser uma vantagem. Para equipes que desejam maior centralização da autoridade do painel na própria trilha backend-gateway, a distância entre leitura rápida e atualização autoritativa precisa ser observada com muito cuidado.

C.5 DOSSIÊ MATERIALIZE

C.5.1 Papel da stack dentro do repositório

Materialize é o backend que mais se aproxima do ideal teórico de manutenção incremental do próprio estado derivado. Sua presença no repositório cumpre função quase metodológica: impedir que o benchmark se reduza a comparação entre três variantes de serving quente e, ao mesmo tempo, testar se uma arquitetura incremental mais “pura” consegue transferir essa elegância conceitual para o plano operacional.

C.5.2 O que a stack parece fazer de forma singular

A singularidade de Materialize não está em liderar p95, mas em oferecer um caminho plausível para desacoplar utilidade inicial do painel e custo integral da consulta. O corpus sugere que o frontend consegue tornar-se útil relativamente cedo mesmo quando as consultas nativas permanecem caras. Isso é uma pista importante de valor arquitetural, porque aponta para uma forma diferente de organizar a experiência near-real-time: menos dependente de serving quente puro e mais dependente de um estado incrementalmente sustentado próximo ao stream.

C.5.3 O que limita a maturidade prática da trilha

Ao mesmo tempo, a rodada canônica final registra alta volatilidade de health e cobertura muito baixa nos snapshots finais de debug. Isso mostra que a trilha incremental ainda não estava operando com a mesma estabilidade prática das melhores execuções de ClickHouse ou Pinot. Há aqui uma distinção crucial entre potência conceitual e prontidão de engenharia. Materialize parece dizer algo importante sobre a forma correta de modelar o problema, mas ainda não entrega, neste corpus, a mesma robustez operacional da solução consultiva mais madura.

C.5.4 Interpretação prática

Materialize é a stack que mais interessa quando o objetivo é empurrar o repositório para um debate tecnicamente mais sofisticado sobre incremental view maintenance, temporalidade e autoridade do snapshot. É menos convincente, por ora, como escolha imediata para o workload já medido. Em termos de pesquisa aplicada, isso é extremamente valioso: a tecnologia obriga o experimento a avançar, mesmo quando não "vence" a bateria final.

C.6 SÍNTESE DOS DOSSIÊS

Lidos em conjunto, os quatro dossiês revelam que o repositório abriga não apenas alternativas de infraestrutura, mas quatro filosofias de aproximação entre evento, estado e visualização. ClickHouse privilegia a excelência do serving quente bem integrado. Druid enfatiza a potência de uma topologia analítica de eventos ao preço de maior exigência operacional. Pinot maximiza a consulta direta com forte protagonismo da borda. Materialize internaliza mais profundamente a lógica incremental, ao custo de maturidade prática ainda incompleta. Para fins de engenharia, essa síntese vale mais do que qualquer ranking isolado.

D APÊNDICE D - GLOSSÁRIO ANALÍTICO DE MÉTRICAS, COMPONENTES E PARADIGMAS

D.1 RAZÃO DESTE GLOSSÁRIO EXPANDIDO

Relatórios longos sobre sistemas orientados a eventos e backends near-real-time tendem a acumular terminologia técnica cuja interpretação varia conforme a tradição de origem. O mesmo termo pode significar coisas diferentes em engenharia de dados, bancos analíticos, sistemas interativos e arquitetura de produto. Este glossário expandido não pretende substituir literatura formal, mas estabilizar o vocabulário efetivamente utilizado neste relatório. Essa estabilização é importante porque o argumento do documento depende fortemente de distinções finas entre conceitos próximos, porém não equivalentes.

D.2 EVENTO CANÔNICO

No contexto deste relatório, evento canônico não é qualquer fato bruto emitido pela aplicação. O termo designa o registro que ocupa o papel de verdade operacional compartilhável dentro do protocolo do benchmark. Mais precisamente, a pesquisa toma como referência o fluxo de entries contábeis normalizadas, e não o evento comercial original ainda heterogêneo. Essa escolha é crítica porque evita que cada backend seja comparado sobre uma interpretação própria do domínio. O evento canônico, portanto, não é apenas um artefato técnico; é a condição de comparabilidade semântica do experimento.

D.3 LEDGER

Ledger, no sentido empregado aqui, é a trilha ordenada, auditável e reprocessável de lançamentos contábeis que representa a história financeira relevante do sistema. Sua importância vai além da persistência. O ledger funciona como eixo temporal, jurídico e metodológico do benchmark. Sem ele, não haveria replay confiável, avaliação consistente de corretude nem possibilidade de distinguir ganho de performance de relaxamento semântico indevido.

D.4 DUPLA ENTRADA

Dupla entrada é o princípio segundo o qual cada fato econômico relevante precisa ser refletido por lançamentos coerentes de débito e crédito, preservando equilíbrio entre classes patrimoniais e de resultado. No benchmark, esse princípio atua como restrição mínima de validade. Não se trata de detalhe contábil externo ao sistema. Trata-se de condição que delimita o que conta como estado derivado aceitável. Um backend que performasse melhor às custas de quebrar a disciplina de dupla entrada não poderia ser considerado tecnicamente superior dentro do problema definido pelo relatório.

D.5 READ MODEL

Read model designa a representação derivada especificamente moldada para servir consultas, painéis e interfaces. O relatório insiste nesse conceito porque as quatro stacks não operam sobre o mesmo tipo de proximidade entre stream e read model. Em algumas, o estado consultável é servido como camada quente sobre dados já materializados. Em outras, o read model participa de uma topologia incremental mais íntima do fluxo de eventos. A distinção importa porque altera o custo de atualização, de replay, de filtragem e de reconciliação visual.

D.6 HOT-ANALYTIC

O termo hot-analytic é usado para designar motores cujo ponto forte é servir com baixa latência um estado analítico já disponível e suficientemente quente. A ênfase está menos na manutenção incremental fina do próprio modelo derivado e mais na excelência de consulta sobre tabelas, segmentos ou índices já prontos para serving. ClickHouse, Druid e Pinot pertencem a essa família, ainda que com internals muito diferentes. A utilidade do termo está em separar essa classe de sistemas de arquiteturas incremental-streaming mais profundas.

D.7 INCREMENTAL-STREAMING

Incremental-streaming designa, neste relatório, arquiteturas em que o processo de manter o estado derivado próximo ao stream é parte central do próprio backend, e não apenas tarefa distribuída entre aplicação, gateway e cache. O caso emblemático do corpus é Materialize. O valor do termo está em indicar uma diferença ontológica no tipo de promessa arquitetural: não se trata apenas de responder rápido sobre um estado pronto, mas de reduzir estruturalmente o custo de manter esse estado em convergência contínua com o fluxo.

D.8 INCREMENTAL VIEW MAINTENANCE

Incremental view maintenance é o campo que estuda como atualizar visões derivadas sem recomputá-las integralmente após cada mudança na base de fatos. No contexto do relatório, o conceito serve para qualificar uma ambição técnica: aproximar o estado consultável de um regime em que diferenças, e não recomputações totais, sejam o principal mecanismo de atualização. A relevância para dashboards near-real-time é direta: quanto mais cara for a recomputação total, maior o interesse por manutenção incremental coerente.

D.9 RETRATAÇÃO

Retratação é a remoção, invalidação ou compensação de contribuição anterior no estado derivado. Em sistemas financeiros, devoluções, cancelamentos, estornos e correções tardias frequentemente assumem essa forma lógica. O conceito é importante porque separa workloads monotônicos de workloads que exigem tratamento explícito de diferença negativa. Boa parte da sofisticação atribuída a sistemas incrementais só se justifica plenamente quando retratações entram no cenário experimental.

D.10 SNAPSHOT AUTORITATIVO

Snapshot autoritativo é a representação do estado que o sistema assume como referência final confiável para exibição, reconciliação e decisão. Ele se opõe a atualizações locais provisórias, projeções parciais ou sinais de progresso ainda não plenamente reconciliados. Em dashboards operacionais, a noção de autoridade é crucial. Um sistema pode ser rápido em mostrar algo; a pergunta decisiva é quando esse algo pode ser tratado como suficiente para orientar ação real.

D.11 CONTINUIDADE VISUAL

Continuidade visual é a propriedade pela qual o frontend evita descontinuidades bruscas, listas vazias espúrias, oscilações severas de agregados ou alternância frequente entre estados semânticos incompatíveis durante a convergência. O termo é usado para lembrar que experiência percebida não depende apenas de velocidade, mas de estabilidade do caminho entre evento e interface. Um painel tecnicamente eventual, porém visualmente errático, pode ser menos útil do que um painel um pouco mais lento, porém semanticamente estável.

D.12 FRONTEND_TIME_TO_FIRST_MEANINGFUL_STATE_MS

Essa métrica representa o intervalo até que o frontend alcance um estado considerado minimamente informativo e operacionalmente útil. Ela não se confunde com o primeiro pacote websocket nem com a primeira resposta HTTP. Seu valor teórico está em medir um limiar semântico de utilidade, não apenas um marco infraestrutural de atividade. Por isso ela é uma das métricas centrais do relatório.

D.13 SNAPSHOT_RATE_PER_SECOND

Essa métrica mede a cadência com que snapshots autoritativos são percebidos na interface ou no gateway, conforme o pipeline de coleta. Sua importância reside em capturar a frequência da reancoragem do painel em um estado efetivamente consolidado. Dois sistemas podem ter tempos iniciais parecidos e, ainda assim, divergir fortemente na taxa com que atualizam o estado autoritativo depois desse primeiro instante.

D.14 ENTRY_RATE_PER_SECOND

A métrica expressa a taxa percebida de propagação de eventos do tipo `entry.created` na trilha de observação usada pelo benchmark. Ela ajuda a distinguir casos em que o sistema é bom em refletir o fluxo elementar de eventos daqueles em que a materialização autoritativa do painel continua lenta. Em conjunto com `snapshot_rate_per_second`, permite separar vivacidade do feed e autoridade do estado derivado.

D.15 HEALTH TIMELINE

Health timeline designa a série temporal de observações de saúde dos componentes monitorados. No relatório, essa trilha é importante porque registra aquecimento, transições de estado e episódios de indisponibilidade parcial. Sua presença impede que a discussão trate readiness como abstração implícita. Em vez disso, readiness entra no corpus como dado observável, comparável e interpretável.

D.16 ROUND CANÔNICA

Rodada canônica é a execução escolhida como representante principal de cada backend no cenário `report-conclusion`. Nesta versão do relatório, a canonicidade não significa perfeição absoluta; significa a execução final mais adequada para a comparação principal, preservando ao mesmo tempo as rodadas auxiliares e substituídas como evidência

diagnóstica. O conceito é importante porque explicita o critério de seleção em vez de ocultá-lo.

D.17 RODADA SUBSTITUÍDA

Rodada substituída é a execução preservada no corpus, mas não usada como representante principal de um backend por ter sido superada por execução posterior mais estável ou mais completa. O caso mais expressivo no corpus é a tentativa anterior de ClickHouse com falhas massivas na superfície SQL. Manter a rodada substituída no artigo é metodologicamente valioso porque torna visível o processo de estabilização do experimento.

D.18 RODADA DE VALIDAÇÃO

Rodadas de validação são execuções auxiliares destinadas a verificar readiness, SQL ou viabilidade de trilhas específicas antes da bateria conclusiva. Elas não possuem o mesmo peso comparativo das rodadas canônicas, mas preservam conhecimento crítico sobre o amadurecimento do protocolo. Em um artigo aprofundado, essas rodadas ajudam a explicar por que certos resultados finais foram possíveis e quais problemas precisaram ser superados para alcançá-los.

D.19 CUSTO OPERACIONAL

Custo operacional é o conjunto de fricções relacionadas a bootstrap, readiness, coordenação de serviços, observabilidade, repetibilidade, diagnóstico e consumo de infraestrutura. O relatório insiste nesse conceito porque muito do que diferencia as stacks não aparece apenas no tempo de query, mas no esforço necessário para fazer a trilha inteira entrar em regime reproduzível. Tratar esse custo como ruído seria empobrecer a comparação.

D.20 CUSTO COGNITIVO

Custo cognitivo designa a complexidade intelectual e de modelagem exigida da equipe para usar um backend de forma correta e sustentável. No corpus, ele aparece de formas diferentes: modelagem incremental e restrições semânticas mais exigentes em Materialize; coordenação entre projeção local e estado autoritativo em Pinot; disciplina de topologia e readiness em Druid; desenho de read model e integração periférica em

ClickHouse. Embora não seja reduzido a um único número, é uma variável real de arquitetura.

D.21 SISTEMA COMPLETO

Ao falar em sistema completo, o relatório recusa a ideia de que o backend seja unidade isolada suficiente de análise. Sistema completo é o arranjo que liga produtor, stream, writer canônico, backend, API, gateway e frontend. O termo é central porque resume a principal contribuição metodológica do trabalho: a unidade de benchmark não é apenas o banco, mas a cadeia que transforma evento em estado utilizável por decisão gerencial.

D.22 ENCERRAMENTO DO GLOSSÁRIO

Este glossário expandido existe para que a leitura do relatório não dependa de suposições tácitas sobre termos fundamentais. Ao estabilizar o vocabulário, ele também reforça a ideia central do trabalho: comparar backends para dashboards near-real-time exige mais do que medir consultas; exige definir com precisão que tipo de verdade operacional, que tipo de autoridade visual, que tipo de custo e que tipo de sistema estão efetivamente em jogo.

E APÊNDICE E - PROGRAMA DE RERUNS, CENÁRIOS E EXTENSÕES DO BENCHMARK

E.1 OBJETIVO DESTE APÊNDICE

O corpus atual é suficientemente forte para sustentar comparação técnica séria, mas ainda não esgota o programa experimental que ele próprio inaugura. Este apêndice organiza, de forma mais operacional, o que deveria ser repetido, ampliado ou tensionado em iterações futuras. Seu valor não está apenas em sugerir "mais testes", e sim em estruturar uma agenda específica de reruns e novos cenários que preserve compatibilidade com o protocolo já construído.

E.2 FAMÍLIA DE RERUNS 1: DUPLA CONFORMAÇÃO DAS RODADAS CANÔNICAS

O primeiro bloco de reruns deve repetir a bateria canônica final com o protocolo atual, o mesmo orçamento de fases e o mesmo contrato funcional. O propósito aqui não é descobrir um workload novo, mas medir estabilidade de repetição. Essa dupla conformação é especialmente importante porque o corpus preserva sinais claros de fragilidade anterior: uma rodada substituída de ClickHouse com falhas massivas, rodadas de validação de Materialize com erros SQL e rodada canônica de Druid ainda com falhas em parte das superfícies.

Nessa família, o objetivo principal não é reinventar a metodologia. É confirmar se as trajetórias estáveis finais realmente se sustentam quando executadas novamente. Se ClickHouse repetir a rodada limpa e rápida, o artigo ganha muito em força conclusiva. Se Druid resolver `filtered_entries` e reduzir as falhas iniciais, melhora-se a comparabilidade SQL. Se Materialize mantiver corretude e reduzir volatilidade, sua posição prática sobe. Se Pinot mantiver sua superioridade consultiva e ampliar cadência autoritativa, a interpretação de sua trilha fica ainda mais rica.

E.3 FAMÍLIA DE RERUNS 2: VARIAÇÃO CONTROLADA DE CARDINALIDADE E PAYLOAD

O corpus atual já mostra que diferentes endpoints carregam custos muito distintos conforme o tamanho e a composição do payload. O próximo passo natural é variar cardinalidade, profundidade de filtros e volume do workspace de modo controlado. Em vez de um único painel padrão, o benchmark pode evoluir para pelo menos três níveis de pressão sobre o estado derivado: leve, intermediário e denso.

No nível leve, o objetivo é observar o comportamento em dashboards com pouca carga de agregação e catálogo. No nível intermediário, preserva-se cenário próximo ao atual. No nível denso, ampliam-se filtros compostos, volume do workspace, amplitude de catálogos e peso dos payloads. Esse desenho mostraria se a vantagem relativa de cada stack é estável ou se ela depende fortemente do tamanho do estado exibido.

E.4 FAMÍLIA DE RERUNS 3: REPLAY E REPROCESSAMENTO

Uma lacuna importante do corpus atual é a ausência de uma bateria especificamente centrada em replay. Isso é relevante porque o problema arquitetural do relatório não se limita ao steady state; ele inclui a capacidade de reconstruir estado confiável quando a trilha precisa ser reprocessada. Um cenário de replay deveria medir, pelo menos, tempo até health verde, tempo até primeiro snapshot útil, tempo até reconciliação de agregados e estabilidade visual do frontend durante a reidratação.

Esse cenário seria particularmente útil para diferenciar mais claramente paradigmas hot-analytic e incremental-streaming. Em teoria, sistemas incrementais podem capturar parte dessa vantagem conceitual com mais nitidez quando o problema deixa de ser consulta pura em estado aquecido e passa a ser reconstrução controlada de estado derivado.

E.5 FAMÍLIA DE RERUNS 4: CORREÇÕES TARDIAS E RETRATAÇÕES

Outra extensão indispensável é introduzir eventos que gerem retratações semânticas reais: cancelamentos, devoluções, estornos, ajustes de classificação e compensações contábeis retroativas. O corpus presente preserva corretude mínima, mas ainda não força o sistema a lidar de maneira intensa com a negativação incremental de contribuições anteriores. Esse é precisamente o tipo de situação em que a teoria de incremental view

maintenance fica mais interessante e em que a robustez do dashboard pode ser testada além do monotônico simples.

Em termos de artigo, um cenário assim permitiria responder perguntas que hoje ainda permanecem abertas. O custo de snapshots autoritativos cresce muito quando o domínio exige retratação? As stacks hot-analytic perdem mais terreno porque precisam recompor mais estado na periferia? Materialize ganha vantagem prática quando a complexidade temporal deixa de ser periférica e passa a ser central? Essas perguntas justificam por si sós uma segunda grande fase do benchmark.

E.6 FAMÍLIA DE RERUNS 5: ROBUSTEZ DE OBSERVABILIDADE E COMPLETUDE DE DEBUG

O programa experimental futuro deveria tratar a completude dos snapshots finais de debug como critério explícito de qualidade da rodada. Uma bateria em que a stack responde rápido, mas não deixa rastro final suficientemente auditável, é cientificamente mais fraca. Por isso, reruns focados em observabilidade devem existir como classe própria. Eles podem verificar, por exemplo, se todos os endpoints de debug permanecem consistentes ao fim da rodada, se os payloads finais preservam campos críticos e se a camada de health timeline consegue reconstruir fielmente transições de readiness.

Esse tipo de cenário é menos glamoroso do que p95, mas extremamente importante. Em projetos reais, o backend que a equipe consegue explicar costuma ser mais valioso do que o backend que apenas produz bons números em uma execução afortunada.

E.7 FAMÍLIA DE CENÁRIOS 6: DEGRADAÇÃO PARCIAL E FALHAS LOCALIZADAS

Uma direção adicional consiste em introduzir degradações localizadas: atraso deliberado em parte da ingestão, indisponibilidade temporária do broker, lentidão controlada de API, perda momentânea de conexão do gateway ou crescimento abrupto de backlog. O interesse aqui não é produzir caos arbitrário, mas observar como cada paradigma se comporta quando a trilha deixa de operar em regime perfeitamente estável. Sistemas gerenciais reais convivem com essas situações com frequência suficiente para que um benchmark de sistema completo não as trate como irrelevantes.

Esse bloco de experimentos também aproxima o trabalho de engenharia resiliente. Ele ajuda a responder se um backend degrada de modo legível, se o painel continua útil sob sofrimento parcial e se a diferença entre “rápido” e “robusto” cresce quando o sistema é pressionado fora da linha ideal.

E.8 FAMÍLIA DE CENÁRIOS 7: EVOLUÇÃO DO PAPEL DO FRONTEND

Como o corpus já mostrou que o frontend é parte constitutiva do fenômeno observado, uma agenda madura de benchmark deve incluir cenários em que políticas de frontend sejam alteradas explicitamente. Isso inclui variar critérios de estado significativo, frequência de reconciliação, agressividade de projeção local, estratégia de fallback e política de tratamento de snapshot vazio ou atrasado. O objetivo é medir quanto da vantagem de cada stack está no backend e quanto está no modo como o frontend negocia com ele a temporalidade do estado exibido.

Pinot é o caso mais óbvio para esse tipo de experimento, mas não o único. ClickHouse e Druid também podem revelar sensibilidades relevantes, e Materialize pode mostrar quanto valor incremental de fato migra do cliente para o backend quando a configuração está mais madura.

E.9 COMO RELATAR ESSES CENÁRIOS SEM PERDER AUDITABILIDADE

À medida que o programa experimental crescer, o artigo precisará continuar fiel ao princípio que justifica sua força atual: cada afirmação importante deve poder ser rastreada a artefatos persistidos. Isso significa que novos cenários não devem ser adicionados se a instrumentação correspondente não estiver preparada. O relatório não precisa relatar tudo ao mesmo tempo; precisa relatar bem o que foi realmente medido.

Uma forma defensável de evoluir a escrita é manter sempre três camadas de evidência. A primeira é a camada canônica, com corpus principal e comparação transversal. A segunda é a camada auxiliar, com rodadas diagnósticas e de validação. A terceira é a camada exploratória, com cenários novos ainda sem massa estatística suficiente para conclusões fortes, mas já úteis para abrir o debate técnico. Essa estratificação evita tanto a simplificação excessiva quanto o acúmulo caótico de dados sem hierarquia.

E.10 CHECKLIST OPERACIONAL PARA RERUNS CANÔNICOS

Como o benchmark depende fortemente de repetibilidade, vale registrar um checklist operacional mais explícito para os reruns de conformação. Primeiro, garantir limpeza completa do ambiente, ausência de resíduos de containers e seleção inequívoca da stack ativa. Segundo, registrar commit, arquivo de ambiente e budgets efetivos antes do início

da coleta. Terceiro, validar não apenas health binário, mas readiness suficiente de frontend, API, gateway e endpoints de debug. Quarto, confirmar a produção ativa de eventos e a materialização mínima do ledger antes de entrar na janela principal. Quinto, ao final, capturar integralmente todos os snapshots finais previstos antes do desligamento da stack.

Esse checklist é simples, mas sua explicitação é importante porque o próprio corpus já mostrou o quanto resultados muito diferentes podem surgir de pequenas diferenças na preparação. Documentá-lo no relatório reduz a dependência de memória tácita e aumenta a chance de reruns verdadeiramente comparáveis.

E.11 ESTRATÉGIA DE PUBLICAÇÃO INCREMENTAL DO CORPUS FUTURO

À medida que novas rodadas forem executadas, o relatório pode crescer sem perder clareza se adotar uma estratégia de publicação incremental. O capítulo principal de resultados deve continuar reservando prioridade às rodadas canônicas mais estáveis. As rodadas auxiliares, de validação e de stress, por sua vez, podem ser absorvidas em apêndices e capítulos diagnósticos específicos, com interpretação mais prudente. Isso evita dois extremos indesejáveis: de um lado, um manuscrito curto demais que apaga contexto; de outro, um manuscrito caótico em que toda execução recebe o mesmo peso analítico.

Em termos práticos, essa estratégia significa que o relatório futuro deve continuar crescendo por camadas. Primeiro, consolida-se a bateria central. Depois, adicionam-se famílias de cenário e reruns com sua própria seção interpretativa. Por fim, quando houver massa suficiente, essas novas famílias podem ser promovidas da condição de evidência auxiliar para condição de evidência principal. Essa governança editorial é importante porque o próprio sucesso do benchmark tende a aumentar rapidamente o volume de artefatos disponíveis.

E.12 FECHO DO APÊNDICE

O programa de reruns e cenários aqui descrito não é mero apêndice opcional. Ele é a continuação lógica do corpus já produzido. O benchmark atingiu um ponto em que a pergunta deixou de ser se vale a pena comparar paradigmas de serving para dashboards orientados a eventos. Vale. A pergunta agora é como expandir essa comparação sem perder rigor, rastreabilidade e utilidade de engenharia. A resposta proposta por este apêndice é simples: repetir onde houve fragilidade, aprofundar onde a teoria pede

pressão adicional e sempre preservar a ligação entre arquitetura, métrica e artefato observável.